

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

(High-Performance E-Commerce Backend Engine)

مشروع البرمجة التفرعية - الفصل الدراسي 2026

مقرر: البرمجة التفرعية مشروع: محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية Stack: Django, Django REST Framework, PostgreSQL, PgBouncer, Redis, Celery, Nginx, Gunicorn, Docker Compose, Locust Report

1. الملخص التنفيذي

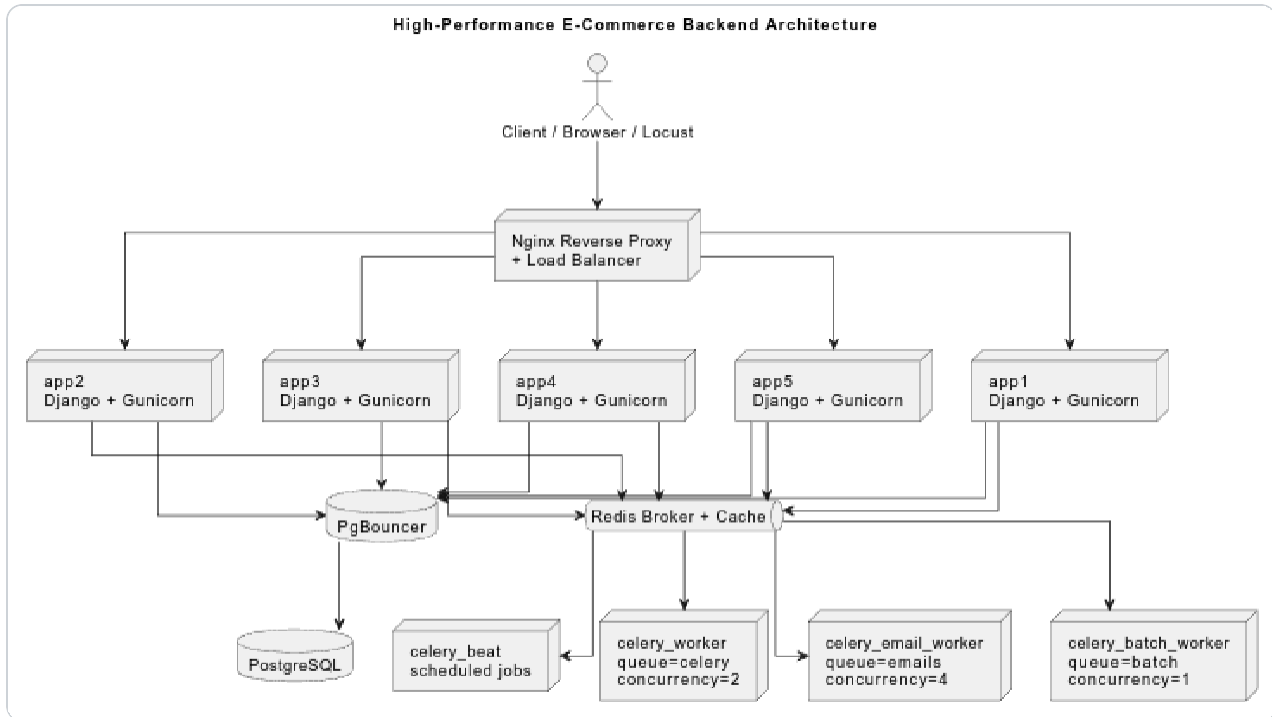
ينفذ هذا المشروع نظام (Backend) للتجارة الإلكترونية مصمم بأسلوب يحاكي بيئات الإنتاج الفعلية، بحيث يدعم عمليات الشراء المتزامنة، ومعالجة المدفوعات، وتحديث أرصدة المحافظ الإلكترونية، وحجز المنتجات، وتنفيذ المهام الخلفية، وتشغيل الوظائف التي تكون على دفعات، إضافة إلى توزيع الأحمال على مكونات النظام المختلفة.

المتطلب	الغرض	الحل المنفذ	الدليل
المتطلب 1	Race condition safety	خمس حالات تزامن باستخدام row locks و state checks	race1_before vs race1_after Locust modes
المتطلب 2	Resource management	Gunicorn gthread workers جميع PgBouncer، تزامن Celery المحدود	app containers x 8 5 workers x 3 threads = 120 request handlers
المتطلب 3	Asynchronous queues	Redis + Celery with separated emails , batch , and celery queues	Sync email/payment waits; async path returns immediately
المتطلب 4	Batch processing	جميع المبيعات اليومية على دفعات باستخدام معالج دفعات مخصص batch worker	Naive task loads all orders; optimized task logs chunks
المتطلب 5	Load distribution	Nginx Least Connections across app1 , app2 , app3 , app4 , and app5	Nginx upstream logs show all five backends

يتكون النظام من خمس حاويات تطبيقات (Containers) تعمل بإطار Django وخادم Gunicorn، وتوجد جميعها خلف خادم Nginx

الذي يتولى توجيه الطلبات وتوزيعها. تُستخدم قاعدة بيانات PostgreSQL لتخزين البيانات الدائمة، بينما يتولى PgBouncer إدارة ضغط اتصالات قاعدة البيانات وتحسين كفاءتها. كما يستخدم Redis كطبقة للتخزين المؤقت وكوسيط رسائل لخدمة Celery، في حين تقوم Celery Workers بمعالجة المهام غير المتزامنة في الخلفية. وأخيراً، تُستخدم أداة Locust لاختبار سلوك النظام والتحقق من أدائه واستقراره تحت ظروف الأحمال المتزامنة والمرتفعة.

2. بنية النظام الحالية



ain runtime containers used by the backen

Container	Purpose
ecommerce_nginx	Public HTTP entry point and load balancer
ecommerce_app1 to ecommerce_app5	Django/Gunicorn API workers
ecommerce_db	PostgreSQL database
ecommerce_pgouncer	Transaction-pooling layer in front of PostgreSQL
ecommerce_redis	Redis cache and Celery broker
ecommerce_celery_worker	General Celery queue
ecommerce_celery_email_worker	Email/background notification queue
ecommerce_celery_batch_worker	Batch-processing queue
ecommerce_celery_beat	Periodic task scheduler

2.1 المتطلبات الوظيفية المنفذة

يلخص هذا القسم المتطلبات الوظيفية الرئيسية التي تم تنفيذها بالفعل في النظام ، قبل الانتقال إلى التركيز على متطلبات التزامن، والسعة التشغيلية ، وإدارة الطوابير، والمعالجة على دفعات ، وتوزيع الأحمال .

إدارة المستخدمين

1. يمكن للمستخدمين إنشاء حسابات جديدة وتسجيل الدخول إلى النظام.
2. يحصل المستخدمون على رمز مصادقة بعد إتمام عملية تسجيل الدخول بنجاح
3. يستطيع العملاء الوصول إلى وظائف التسوق، وسلة المشتريات، وإدارة الطلبات
4. يمتلك ال Admins صلاحيات موسعة لإدارة المنتجات والطلبات
5. يسمح للمسؤولين باستخدام التدفقات المخصصة للعملاء فقط، مثل إتمام الشراء عبر سلة المشتريات أو حجز المنتجات.

إدارة المنتجات

1. يمكن للعملاء استعراض المنتجات النشطة المتاحة للبيع.
2. يمكن للعملاء الاطلاع على التفاصيل الكاملة للمنتجات.
3. يستطيع المسؤولون إنشاء منتجات جديدة.
4. يستطيع المسؤولون تعديل بيانات المنتجات.
5. يستطيع المسؤولون حذف المنتجات أو تعطيلها.
6. يستطيع المسؤولون إعادة تزويد المنتجات بالمخزون (Restocking).
7. يستطيع المسؤولون عرض سجلات المخزون.
8. يتم تتبع كميات المخزون الخاصة بكل منتج بشكل مستمر.
9. يتم تسجيل جميع التغييرات التي تطرأ على المخزون في سجلات مخصصة.

إدارة سلة المشتريات

1. يمكن للعملاء عرض محتويات سلة المشتريات الخاصة بهم.
2. يمكن للعملاء إضافة المنتجات إلى السلة.
3. يمكن للعملاء تعديل كميات المنتجات الموجودة في السلة.
4. يمكن للعملاء إزالة المنتجات من السلة.
5. يمكن للعملاء تفريغ السلة بالكامل.
6. تتحقق عمليات السلة من وجود المنتج وصلاحيه الكمية المطلوبة وتوافر المخزون .
7. يمنع المسؤولون من استخدام وظائف سلة المشتريات.

إدارة الطلبات

1. يمكن للعملاء إتمام عملية الشراء (Checkout) اعتماداً على محتويات سلة المشتريات.
2. تؤدي عملية إتمام الشراء إلى إنشاء طلب جديد استناداً إلى العناصر الموجودة في السلة.
3. يتم خصم الكميات المطلوبة من مخزون المنتجات عند تنفيذ عملية الشراء.
4. يتم إنشاء عناصر الطلب (Order Items) المرتبطة بالطلب.

5. يتم إنشاء سجلات الدفع الخاصة بالطلب.
6. يمكن للعملاء عرض طلباتهم الخاصة.
7. يمكن للعملاء الاطلاع على تفاصيل طلباتهم.
8. يمكن للمسؤولين عرض جميع الطلبات الموجودة في النظام.
9. يمكن للعملاء إلغاء الطلبات الخاصة بهم عندما تكون في حالة تسمح بالإلغاء.
10. يمكن للمسؤولين تحديث حالة الطلبات.
11. يتم منع الانتقالات غير الصحيحة بين حالات الطلب المختلفة.

نظام الدفع

1. يمكن للعملاء تنفيذ عمليات الدفع الخاصة بطلباتهم فقط.
2. يمكن تحديث حالة الدفع إلى مكتمل بعد نجاح عملية الدفع.
3. يتم منع معالجة عمليات الدفع المكررة للطلب نفسه.
4. لا يمكن معالجة الدفع للطلبات الملغاة.
5. يدعم النظام إتمام الشراء باستخدام المحفظة الإلكترونية.
6. يتم التحقق من رصيد المحفظة قبل تنفيذ عملية الشراء.
7. يتم خصم قيمة الطلب من رصيد المحفظة بعد نجاح عملية الدفع عبر المحفظة.
8. يؤدي عدم كفاية رصيد المحفظة إلى منع إتمام عملية الشراء.

حجز المنتجات

1. يمكن للعملاء حجز كمية من مخزون المنتج بصورة مؤقتة.
2. تؤدي عملية حجز المنتج إلى إنشاء قفل حجز مرتبط بالمنتج والكمية المطلوبة.
3. يمكن تحرير أقفال الحجز المنتهية الصلاحية بعد انتهاء مدة الحجز.
4. يتم منع الحجز الزائد باستخدام آلية قفل الصفوف على مستوى قاعدة البيانات.
5. لا يسمح للAdmins باستخدام وظيفة حجز المنتجات.

التقييمات والمراجعات

1. يمكن للعملاء عرض تقييمات ومراجعات المنتجات.
2. يمكن للعملاء إنشاء مراجعات وتقييمات للمنتجات.
3. لا يُسمح للعميل بتقييم منتج ما إلا إذا كان قد قام بشرائه مسبقاً.
4. لا يمكن للعميل إرسال أكثر من مراجعة واحدة للمنتج نفسه.

المعالجة غير المتزامنة والمهام الخلفية

1. يتم إرسال رسائل تأكيد الطلبات عبر البريد الإلكتروني بصورة غير متزامنة.
2. يتم إرسال رسائل إلغاء الطلبات عبر البريد الإلكتروني بصورة غير متزامنة.
3. يمكن تنفيذ عمليات الدفع باستخدام المحفظة الإلكترونية بصورة غير متزامنة من خلال Celery.
4. يدعم النظام معالجة المبيعات اليومية على شكل دفعات.
5. تدعم المعالجة على دفعات تقسيم البيانات إلى دفعات أو أجزاء صغيرة لتحسين الكفاءة وقابلية التوسع.

6. يمكن تنفيذ عمليات تنظيف دورية لإزالة أقفال حجز المنتجات المنتهية الصلاحية.

متطلبات التزام واتساق البيانات

1. يجب أن تمنع عملية إتمام الشراء بيع كميات تتجاوز المخزون المتاح.
2. يجب أن تمنع عمليات الدفع عبر المحفظة الإلكترونية الإنفاق المزدوج على الرصيد نفسه.
3. يجب أن تمنع آلية معالجة المدفوعات تنفيذ عملية الدفع نفسها أكثر من مرة.
4. يجب أن تحافظ حالات التنافس بين الإلغاء والدفع على الحالة الصحيحة والمنطقية للطلب.
5. يجب أن تمنع آلية حجز المنتجات حجز كميات تتجاوز المخزون المتاح.

3. المتطلب 1: حماية البيانات المشتركة من التضارب

نظرة عامة

لا يقتصر النظام المنفذ على كونه واجهة برمجية بسيطة لتنفيذ عمليات الإنشاء والقراءة والتحديث والحذف (CRUD API)، بل يتضمن تطبيقات عملية وآليات صريحة للتحكم في التزام بهدف معالجة وإثبات التعامل الصحيح مع حالات التنافس. ويشمل النظام خمس حالات رئيسية من حالات التنافس التي قد تظهر في أنظمة التجارة الإلكترونية:

1. إتمام عمليات الشراء المتزامنة ومنع البيع الزائد للمخزون (Concurrent Checkout / Overselling).
2. عمليات الدفع بالمحفظة الإلكترونية ومنع الإنفاق المزدوج للرصيد (Wallet Checkout / Double Spending).
3. منع معالجة عملية الدفع الواحدة أكثر من مرة (Double Payment Processing).
4. التعامل مع حالة إلغاء الطلب أثناء تنفيذ عملية الدفع (Cancel Order While Payment Is Processing).
5. حجز المنتجات ومنع الحجز الزائد عن الكميات المتاحة (Product Reservation / Over-Reservation).

3.1 الهدف

يهدف هذا المتطلب إلى إثبات أن البيانات المشتركة في أنظمة التجارة الإلكترونية يمكن أن تتعرض لعدم الاتساق أو الفشل عند الوصول إليها بصورة متزامنة من قبل عدة مستخدمين أو عمليات في الوقت نفسه، ثم توضيح الكيفية التي يستخدم بها النظام الخلفي المنفذ آليات التحكم في التزام لمنع هذه المشكلات والحفاظ على سلامة البيانات واتساقها.

ولا يقتصر المشروع على التحقق من حالة واحدة مرتبطة بالمخزون، بل يقوم بدراسة واختبار خمس حالات عملية ومحددة من حالات التنافس التي تُعد من أكثر المشكلات شيوعاً وحساسية في الأنظمة التجارية ذات الأحمال المرتفعة، وذلك بهدف ضمان صحة العمليات واستقرار النظام تحت ظروف التشغيل المتزامنة.

Case	Problem	Demo / Risk Endpoint	Protected Endpoint
------	---------	----------------------	--------------------

Case 1	Concurrent checkout can oversell stock	POST /api/orders/race-demo/	POST /api/orders/checkout/
Case 2	Wallet checkout can double-spend balance	POST /api/orders/blocking-wallet-checkout/	POST /api/orders/checkout-wallet-async/
Case 3	Same order can be paid twice	POST /api/orders/<id>/process-payment-unsafe/	POST /api/orders/<id>/process-payment/
Case 4	Paid order can be cancelled during payment transition	POST /api/orders/<id>/cancel-unsafe/	POST /api/orders/<id>/cancel/
Case 5	Product reservations can exceed available stock	POST /api/products/<id>/reserve-unsafe/	POST /api/products/<id>/reserve/

3.2 تقنيات المزامنة المستخدمة

Technique	Used In	Reason
<code>transaction.atomic()</code>	Checkout, wallet checkout, payment, cancel, restock, reservation	Keeps related writes all-or-nothing
<code>select_for_update()</code>	Product rows, user wallet row, order row, payment row	Serializes critical updates on shared rows
Consistent lock ordering	Product rows sorted by id during checkout	Reduces deadlock risk
State validation	Payment and cancellation endpoints	Prevents invalid transitions such as paying a cancelled order
Optimistic version field	Product helper logic	Detects lower-risk inventory conflicts without holding locks

3.3 تفاصيل تنفيذ آليات القفل

يستخدم النظام الخلفي استراتيجيتين مختلفتين للقفل، وذلك اعتماداً على مستوى المخاطر المرتبط بالعملية المنفذة.

القفل التفاولي (Optimistic Locking)

يُستخدم optimistic locking في الحالات التي يكون فيها احتمال حدوث التعارض منخفضاً، وتكون إعادة تنفيذ العملية عند حدوث التعارض أمراً آمناً ومقبولاً.

1. `apps/cart/models.py` - `CartItem.version` field

يُعدّ الحقل `Version` من نوع `Integer` في جدول `CartItem` الأساس الذي تعتمد عليه آلية التحكم بالتزامن. فمع كل عملية تحديث، يتم زيادة قيمة هذا الحقل بمقدار واحد. إذا قام طلبان بقراءة القيمة `version = 3` في الوقت نفسه، فسيحاول كلاهما تنفيذ التحديث باستخدام الشرط `WHERE version = 3`. ينجح الطلب الأول فقط ويؤثر على صف واحد، بينما يحصل الطلب الثاني على صفر صفوف متأثرة، مما يؤدي إلى اكتشاف التعارض (`Conflict`) بشكل فوري.

2. `apps/products/models.py` - `Product.version` field

تُستخدم الآلية ذاتها في نموذج `Product`. فكل عملية تؤثر على مخزون المنتج (`Stock`) تؤدي إلى زيادة قيمة حقل `version`. وبذلك يصبح من الممكن اكتشاف أي تعديل متزامن على بيانات المخزون من خلال التحقق من قيمة الإصدار قبل تنفيذ عملية التحديث.

3. `apps/cart/services.py` - `_try_add_to_cart_optimistic()` and `add_to_cart()`

عند تنفيذ عملية إضافة منتج إلى سلة المشتريات (`Add to Cart`)، تقوم الخدمة أولاً بقراءة سجل `CartItem` دون استخدام قفل على مستوى قاعدة البيانات. بعد ذلك تحاول تنفيذ التحديث باستخدام الشرط `captured_version` الذي يمثل قيمة الإصدار الذي تمت قراءته سابقاً إذا لم يتم تحديث أي صف، فإن ذلك يعني أن عملية أخرى قامت بتعديل السجل في أثناء التنفيذ، وبالتالي يتم اعتبار الحالة تعارضاً

(Conflict) في هذه الحالة تُعيد الدالة المساعدة (Helper Function) إشارة تدل على وجود تعارض، وتقوم الدالة (add_to_cart) بإعادة المحاولة تلقائياً حتى خمس مرات كحد أقصى ويتم تطبيق التراجع الأسّي (Exponential Backoff) بين المحاولات وذلك لتقليل احتمالية استمرار التصادم بين الطلبات المتزامنة.

4. `apps/products/services.py` - `deduct_stock_optimistic()` and `deduct_stock_with_retry()`

قبل تنفيذ عملية الشراء النهائية (Checkout)، يتم إجراء فحص أولي للمخزون دون استخدام أقفال على مستوى قاعدة البيانات. تقوم العملية بقراءة بيانات المنتج ثم محاولة تنفيذ التحديث باستخدام الشرط: `WHERE version = <captured_version> AND stock >=` quantity وهذا الشرط يحقق هدفين في الوقت نفسه:

- التأكد من أن نسخة بيانات المنتج لم تتغير منذ قراءتها.
- التأكد من أن كمية المخزون المتاحة لا تزال كافية لتلبية الطلب.

إذا كان عدد الصفوف المتأثرة صفراً، فإن ذلك يشير إلى أحد احتمالين: حدوث تعارض في الإصدار (Version Conflict) نتيجة تعديل متزامن أو عدم توفر كمية كافية من المخزون. في كلتا الحالتين تقوم آلية إعادة المحاولة (Retry Wrapper) بإعادة تنفيذ العملية حتى خمس مرات كحد أقصى.

القفْل التّشاوُمي (Pessimistic Locking `SELECT FOR UPDATE`)

يُستخدم القفل التّشاوُمي في الحالات التي تكون فيها احتمالية التعارض مرتفعة أو تكون نتائج التعارض غير قابلة للتراجع.

5. `apps/orders/services.py` - `create_order_from_cart()`

يتم قفل جميع سجلات المنتجات دفعة واحدة باستخدام `select_for_update()` مع ترتيبها حسب المعرّف (ID) لتقليل احتمالية حدوث الاختناق المتبادل (Deadlock). وأي عملية شراء أخرى تتعامل مع المنتجات نفسها تنتظر حتى انتهاء المعاملة الحالية.

6. `apps/orders/services.py` - `checkout_with_wallet()`

يتم أولاً قفل سجل المستخدم لحماية رصيد المحفظة من الإنفاق المزدوج عبر الطلبات أو النوافذ المتعددة، ثم يتم قفل جميع سجلات المنتجات. ويمنع ذلك المستخدم من إنفاق مبلغ يتجاوز رصيده المتاح.

7. `apps/orders/services.py` - `process_payment()`

يتم قفل كلّ من سجل الطلب (Order) وسجل الدفع (Payment)، مما يمنع معالجة عملية الدفع نفسها أكثر من مرة، وهو خطأ مالي غير مقبول.

8. `apps/orders/services.py` - `cancel_order()`

يتم قفل سجل الطلب لمنع حدوث تعارض بين طلب الإلغاء وطلب معالجة الدفع على الطلب نفسه في الوقت ذاته.

9. `apps/orders/services.py` - `update_order_status()`

يتم قفل سجل الطلب أثناء تحديث حالته من قبل المسؤول، لمنع التحديثات الإدارية المتزامنة من إنتاج حالة نهائية غير متسقة.

10. `apps/cart/services.py` - `update_cart_item_quantity()`

يتطلب تعيين الكمية إلى قيمة محددة (N) تسلسلاً صارماً للعمليات، لذلك يُستخدم القفل التّشاوُمي لضمان وضوح النتيجة النهائية.

11. `apps/products/services.py` - `restock_product()`

يتم قفل سجل المنتج أثناء إعادة التزويد بالمخزون لمنع عمليتي تزويد متزامنتين للمنتج نفسه من التسبب في فساد عدد الوحدات الإجمالي.

12. `apps/products/services.py` - `create_order_lock()`

يتم قفل سجل المنتج بشكل حصري أثناء إنشاء سجل الحجز المؤقت للمخزون خلال عملية الشراء.

13. `apps/products/services.py` - `deduct_stock_pessimistic()`

تُستدعى هذه الدالة أثناء عملية الشراء بعد أن يكون سجل المنتج مقفلاً مسبقاً. وتقوم بخصم المخزون بأمان وإنشاء سجل تدقيق في InventoryLog تحت القفل نفسه.

ACID Wrapper `transaction.atomic()`

جميع الدوال السابقة مغلفة أيضاً باستخدام `transaction.atomic()`، مما يعني أن جميع العمليات المرتبطة تُنفَّذ بالكامل معاً أو يتم التراجع عنها بالكامل معاً:

1. `create_order_from_cart()` - خصم المخزون، وإنشاء الطلب، وإنشاء سجل الدفع إما أن تنجح جميعها أو يتم التراجع عنها جميعاً.
2. `checkout_with_wallet()` - خصم رصيد المحفظة، وخصم المخزون، وإنشاء الطلب، وإنشاء سجل الدفع تُنفَّذ كوحدة ذرية واحدة.
3. `process_payment()` - لا يمكن تحديث حالة الدفع وحالة الطلب بشكل جزئي.
4. `cancel_order()` - تُنفَّذ عملية الإلغاء كتغيير حالة واحد ومتسق.
5. `deduct_stock_pessimistic()` - يتم تسجيل تعديل المخزون وسجل InventoryLog معاً، مما يضمن بقاء سجل التدقيق متزامناً مع بيانات المخزون.

3.4 الحالة 1: إتمام عمليات الشراء المتزامنة والبيع الزائد للمخزون

شرح الحالة الأولى: تحدث هذه المشكلة عندما يحاول عدة مستخدمين شراء المنتج نفسه الذي يمتلك كمية محدودة في المخزون في الوقت ذاته.

قبل تطبيق آليات المعالجة والحماية، تقوم نقطة الشراء غير الآمنة بقراءة قيمة المخزون دون تطبيق أي قفل على سجل المنتج داخل قاعدة البيانات. ونتيجة لذلك، يمكن لعدة طلبات متزامنة أن تعتمد على القيمة نفسها للمخزون أثناء اتخاذ قرار الشراء.

النتيجة المحتملة: قد يؤدي هذا السلوك إلى انخفاض قيمة المخزون إلى ما دون الصفر، وهو دليل واضح على حدوث البيع الزائد للمخزون، حيث يتم بيع عدد من الوحدات يفوق الكمية المتوفرة فعلياً.

حالة قاعدة البيانات قبل بدء الاختبار:

تم فحص قاعدة البيانات قبل تنفيذ الاختبار للتأكد من الكمية الابتدائية للمخزون، والتي ستستخدم كمرجع للمقارنة بعد انتهاء الاختبار.

Query Query History

```
1 SELECT id, name, stock FROM products_product WHERE stock <= 10 ORDER BY stock;
```

Data Output Messages Notifications

Showing rows: 1 to 5

	id [PK] bigint	name character varying (255)	stock integer
1	4842	Last-Chance iPhone 15 Pro 1TB	-2
2	4844	Flash Sale Sony Camera Kit	3
3	4841	Limited Edition PlayStation 5 BundL	5
4	4843	Exclusive MacBook Pro M3 Max	6
5	4845	Rare Nintendo Switch Bundle	8

تم استخدام أداة Locust، وهي أداة اختبار مبنية بلغة Python تُستخدم لمحاكاة الأحمال المتزامنة، حيث تسمح بإرسال عدد محدد من الطلبات في الوقت نفسه بهدف اختبار سلوك النظام تحت ظروف التنافس

LOCUST

Host: http://nginx:80 Status: STOPPED RPS: 49.41 Failures: 54% NEW RESET

TESTING CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

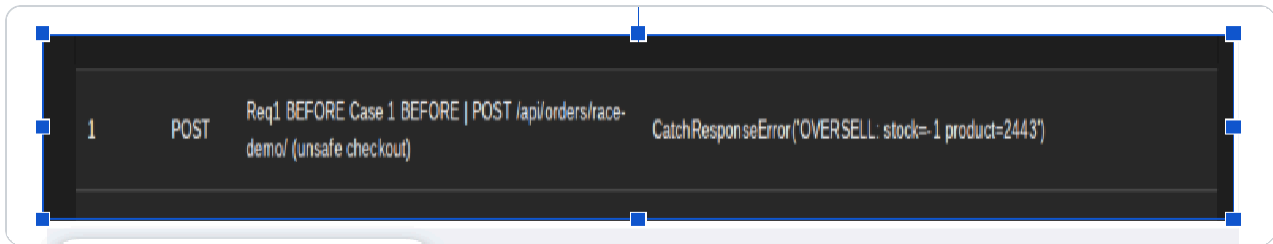
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	70	0	24000	31000	34000	22836.29	4100	33526	52	1.4	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	733	361	280	820	4200	479.99	131	23672	153.63	41.7	20.4

قبل: [POST /api/orders/race-demo/](#)

سلوك **endpoint غير الآمنة**: تم تصميم نقطة النهاية غير الآمنة عمداً لإظهار المشكلة، حيث تقوم بالخطوات التالية:

1. قراءة بيانات المنتج دون استخدام أي آلية قفل (Locking).
 2. الانتظار لمدة 100 ميلي ثانية لمحاكاة فترة معالجة واقعية.
 3. تخفيض قيمة المخزون باستخدام عملية تحديث مباشرة دون فرض حد أدنى يمنع المخزون من النزول إلى الصفر أو إلى قيم سالبة.
- وبسبب غياب آليات الحماية، تستطيع العديد من الطلبات المتزامنة متابعة التنفيذ اعتماداً على معلومات قديمة (Stale Data) حول المخزون، مما يؤدي إلى خصم الكمية أكثر من مرة ودفع المخزون إلى قيم سالبة.

فشل endpoint: أظهرت نتائج الاختبار فشل نقطة النهاية في التعامل مع الطلبات المتزامنة بصورة صحيحة، حيث لم تتمكن من حماية بيانات المخزون من التعديلات المتزامنة، مما أدى إلى ظهور حالة البيع الزائد للمخزون.



حالة قاعدة البيانات بعد انتهاء الاختبار: بعد الانتهاء من تنفيذ الاختبار، تم فحص قاعدة البيانات مرة أخرى ومراجعة قيم المخزون الفعلية. وقد أكدت النتائج وجود المشكلة بالفعل، حيث تبين أن المخزون تعرض لتعديلات متزامنة غير محمية أدت إلى نتائج غير صحيحة، مما يثبت حدوث Overselling. ويُعد ظهور قيم سالبة للمخزون أو انخفاض الكمية إلى ما دون الحد المنطقي دليلاً مباشراً على فشل آلية إدارة المخزون في النسخة غير الآمنة من النظام.

id	name	stock
1	Limited Edition PlayStation 5 Bund...	-2194
2	Last-Chance iPhone 15 Pro 1TB	-48
3	Exclusive MacBook Pro M3 Max	-1778
4	Flash Sale Sony Camera Kit	-1221
5	Rare Nintendo Switch Bundle	-32
306	Limited Edition PlayStation 5 Bund...	-1
307	Last-Chance iPhone 15 Pro 1TB	7
308	Exclusive MacBook Pro M3 Max	-5084
309	Flash Sale Sony Camera Kit	4
310	Rare Nintendo Switch Bundle	3
611	Limited Edition PlayStation 5 Bund...	3
612	Last-Chance iPhone 15 Pro 1TB	2
613	Exclusive MacBook Pro M3 Max	-3821
614	Flash Sale Sony Camera Kit	7
615	Rare Nintendo Switch Bundle	3
916	Limited Edition PlayStation 5 Bund...	4
917	Last-Chance iPhone 15 Pro 1TB	7
918	Exclusive MacBook Pro M3 Max	-145
919	Flash Sale Sony Camera Kit	4
920	Rare Nintendo Switch Bundle	3
1221	Limited Edition PlayStation 5 Bund...	6
1222	Last-Chance iPhone 15 Pro 1TB	8

بعد: سناقش الآن كيفية معالجة هذا النوع من المشكلات ومنع حدوثه في بيئة التشغيل الفعلية.

1. إنشاء الطلبات ومنع البيع الزائد للمخزون (Order Creation / Preventing Overselling)

الأسلوب المستخدم: القفل التشاؤمي (Pessimistic Locking) مع المعاملات الذرية (transaction.atomic())

تم استخدام أسلوب القفل التشاؤمي (Pessimistic Locking) لأن عملية إنشاء الطلب تتضمن تعديل المخزون، والذي يُعد مورداً مشتركاً بين عدد كبير من المستخدمين. ولذلك يتم قفل سجلات المنتجات باستخدام الدالة (select_for_update()) قبل التحقق من المخزون وخصم الكميات المطلوبة.

كما تم استخدام transaction.atomic() لضمان تنفيذ عملية إنشاء الطلب كوحدة ذرية واحدة (Atomic Unit)، بحيث يتم تنفيذ جميع خطوات العملية بنجاح أو التراجع عنها بالكامل في حال حدوث أي خطأ أثناء التنفيذ.

النتيجة الآمنة (Safe Outcome)

- يتم بيع الكمية المتاحة فقط من المنتج.
- يتم رفض أي طلب يؤدي إلى تجاوز المخزون المتوفر (Overselling).
- لا يمكن أن تصبح قيمة المخزون سالبة.
- تبقى بيانات المخزون متسقة وصحيحة حتى في حالات الوصول المتزامن.

آلية تنفيذ عملية الشراء الفعلية (Real Checkout Process): تستخدم عملية الشراء الفعلية قفلاً على سجلات المنتجات بواسطة `select_for_update()`. تبدأ العملية بإنشاء معاملة (Transaction)، ثم يتم قفل جميع سجلات المنتجات المطلوبة. وأثناء استمرار هذه الأقفال، يتم تنفيذ الخطوات التالية بالتتابع:

1. التحقق من توافر الكميات المطلوبة في المخزون.
2. خصم الكميات المطلوبة من مخزون المنتجات.
3. إنشاء سجل الطلب (Order).
4. إنشاء عناصر الطلب (Order Items).
5. إنشاء سجل الدفع (Payment Record).
6. تفرغ سلة المشتريات الخاصة بالمستخدم.
7. إبطال أو تحديث ذاكرة التخزين المؤقت الخاصة بالمنتجات (Product Cache Invalidation).

النتيجة الآمنة المتوقعة: تضمن هذه الآلية أن الطلبات المتزامنة لا تستطيع تعديل مخزون المنتج نفسه في الوقت ذاته. ونتيجة لذلك، يتم منع البيع الزائد للمخزون، والحفاظ على اتساق بيانات المنتجات والطلبات والمدفوعات، وضمان عدم انخفاض المخزون إلى قيم سالبة حتى تحت ظروف الحمل المرتفع والتنفيذ المتزامن للطلبات.

Stock never becomes negative.
Some requests succeed with HTTP 201.
Excess requests are rejected with HTTP 400 when stock is exhausted.

الاختبار بعد تطبيق الحل باستخدام Locust:

 LOCUST

Host
http://nginx:80

Status
STOPPED

RPS
35.1

Failures
15%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	79	0	28000	37000	39000	25109.36	1769	39415	52	3.5	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	257	0	550	4300	13000	986.07	78	14032	418.35	11.7	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	173	0	220	3200	5500	568.32	15	6993	421.5	8.5	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	309	0	240	610	1100	346.18	13	13934	79.63	16.1	0
POST	Req1 AFTER Case 4 cancel while payment processing POST /api/orders/{id}/cancel/	65	0	200	470	2700	259.39	17	2738	53	2.7	0
POST	Req1 AFTER Case 5 POST /api/products/{id}/reserve/ (over-reservation protected)	191	0	190	530	2100	302.81	13	14539	40.14	8.6	0
POST	Req1 AFTER Setup POST /api/cart/add/	712	268	260	740	1400	379.29	12	23061	142.19	33.3	13.6
Aggregated		1786	268	270	4700	31000	1560.52	12	39415	180.01	84.4	13.6

ABOUT

ABOUT

قد فشلت بعض الطلبات أثناء الاختبار لأن مخزون المنتج أصبح فارغاً، ولذلك أعاد النظام استجابة 404 Not Found وفقاً لمنطق الأعمال المطبق، وليس نتيجة وجود خطأ في آلية حماية المحفظة الإلكترونية.

 LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

53.11

Failures

2%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

#

Failures

Method

Name

Message

194

POST

/api/cart/add/

CatchResponseError('Cart add failed: 404')

نتائج الفحص من خلال (Terminal)

REQUIREMENTS TEST REPORT	
Mode: race1_after	
REQ 1 – Race Condition Safety (5 Cases)	
[General] HOT checkouts via EcommerceUser succeeded :	0
[General] Stock-outs blocked :	0
CASE 1 – Concurrent Checkout / Overselling	
BEFORE endpoint :	POST /api/orders/race-demo/ (no lock, 100ms sleep → stock < 0)
AFTER endpoint :	POST /api/orders/checkout/ (SELECT FOR UPDATE on product rows)
Oversell events (stock<0) :	0
Safe checkout ok (201) :	237
Correctly blocked (400) :	20
✓ PROTECTED – pessimistic lock prevented overselling	

عدم وجود أي أرصدة سالبة داخل قاعدة البيانات.

[eбраheim@arch high-performance-ecommerce-backend]\$ docker exec ecommerce_db psql -U ecommerce_user -d ecommerce_db -c "SELECT id, name, stock FROM products_product WHERE stock <= 10 ORDER BY stock;"	
\	
id	name stock
-----+-----+-----	
13686	Limited Edition PlayStation 5 Bundle 0
13689	Flash Sale Sony Camera Kit 0
13690	Rare Nintendo Switch Bundle 0
13991	Limited Edition PlayStation 5 Bundle 0
13992	Last-Chance iPhone 15 Pro 1TB 0
13995	Rare Nintendo Switch Bundle 0
13687	Last-Chance iPhone 15 Pro 1TB 0
13994	Flash Sale Sony Camera Kit 0
13993	Exclusive MacBook Pro M3 Max 0
13688	Exclusive MacBook Pro M3 Max 0
(10 rows)	
>	

3.5 الحالة 2: الدفع بالمحفظة الإلكترونية ومنع الإنفاق المزدوج

تحدث هذه الحالة عندما يتم إرسال عدة طلبات دفع بصورة متزامنة من المحفظة الإلكترونية الخاصة بالمستخدم نفسه.

قبل تطبيق آليات الحماية المناسبة، إذا لم يكن رصيد المحفظة محمياً ضد الوصول المتزامن، فقد تتمكن عدة طلبات من قراءة الرصيد نفسه قبل خصم المبلغ المطلوب منه.

النتيجة المحتملة: قد يتمكن المستخدم من إنفاق الرصيد نفسه أكثر من مرة نتيجة تنفيذ عدة عمليات دفع متزامنة تعتمد على القيمة نفسها للرصيد قبل تحديثها.

مثال توضيحي: إذا كان رصيد المحفظة يساوي 100 وحدة نقدية، وتم إرسال طلبي دفع متزامنين بقيمة 80 وحدة نقدية لكل منهما، فقد تنجح العمليتان معاً في حال عدم وجود آلية حماية مناسبة، لأن كلا الطرفين قد يقرأ الرصيد قبل أن يتم تحديثه أو خصم المبلغ منه.

وفي هذه الحالة قد يتجاوز إجمالي المبلغ المنفق الرصيد الحقيقي المتاح، مما يؤدي إلى مشكلة الإنفاق المزدوج (Double Spending) وإلى فقدان اتساق البيانات المالية. path:

POST /api/orders/blocking-wallet-checkout(double-spend)

يرتبط هذا المسار بعملية الدفع المتزامنة باستخدام المحفظة الإلكترونية. ويمثل هذا المسار إحدى المناطق الحساسة المعرضة لحدوث حالات التنافس، حيث تتنافس عدة طلبات شراء متزامنة على استخدام الرصيد نفسه الخاص بالمستخدم، في الوقت الذي تستغرق فيه محاكاة بوابة الدفع ثلاث ثوانٍ لإتمام المعالجة.

في التنفيذ الحالي، يقوم النظام بحماية هذا المسار من خلال تطبيق آلية قفل على سجل المستخدم قبل التحقق من رصيد المحفظة وخصم المبلغ المطلوب. ونتيجة لذلك، لا يمكن لأكثر من طلب واحد الوصول إلى الرصيد نفسه وتعديله في الوقت ذاته، مما يضمن سلامة البيانات المالية ويمنع حدوث الإنفاق المزدوج.

تُظهر هذه الصورة نتائج اختبار سيناريو الدفع بالمحفظة الإلكترونية ومنع الإنفاق المزدوج (Wallet Checkout / Double Spend) باستخدام أداة Locust، حيث تم إرسال 269 طلباً متزامناً إلى نقطة نهاية الدفع بالمحفظة الإلكترونية.

وتُظهر النتائج وجود عدد كبير من الطلبات التي تم رفضها أو فشلت في التنفيذ، وهو سلوك متوقع وطبيعي في هذا السيناريو، لأن النظام مصمم لمنع عدة طلبات متزامنة من استخدام الرصيد نفسه في الوقت ذاته.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	70	0	24000	31000	34000	22836.29	4100	33526	52	1.4	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	733	361	280	820	4200	479.99	131	23672	153.63	41.7	20.4
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	269	234	150	3400	3800	634.12	11	4094	190.96	16	13.5

كما توضح الصورة أن أداة Locust سجّلت 234 طلباً مرفوضاً ضمن هذا الاختبار، نتيجة ظهور رمز الحالة 409 Conflict. ولا يشير هذا الرمز في هذه الحالة إلى وجود خطأ أو فشل في النظام، بل يدل على أن النظام اكتشف محاولة تعارض (Conflict) أثناء تنفيذ عملية الدفع عبر المحفظة الإلكترونية وقام بمنعها بشكل آمن.

ويعني ذلك أن طلباً آخر كان يحاول استخدام الرصيد نفسه في الوقت ذاته، ولذلك تم رفض الطلب الحالي حفاظاً على سلامة الرصيد ومنع حدوث مشكلة الإنفاق المزدوج

```
234      POST      Req1 BEFORE Case 2 | POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)      CatchResponseError('409 Conflict: wallet double-spend race blocked')
```

تحليل النتائج بعد انتهاء الاختبار

```
CASE 2 – Wallet Checkout / Double Spend
BEFORE endpoint : POST /api/orders/blocking-wallet-checkout/
AFTER endpoint  : POST /api/orders/checkout-wallet-async/
Scenario : concurrent wallet checkouts – user row locked to prevent double spend
Queued (202)           : 35
Blocked – low balance (400) : 234
```

كما تم التقاط صورة توضح الرصيد النهائي للمحفظة الإلكترونية لسببين رئيسيين:

1. ثبات تعرض الرصيد لضغط فعلي نتيجة الطلبات المتزامنة، أي أن عدداً كبيراً من عمليات الشراء قد تم تنفيذه باستخدام المحفظة الإلكترونية خلال الاختبار.

Query Query History	
1 SELECT email, wallet_balance FROM users_user WHERE email LIKE 'locust_%@test.com' ORDER BY wallet_balance ASC LIMIT 10;	
Data Output Messages Notifications	
email character varying (255)	wallet_balance numeric (10,2)
1 locust_79@test.com	1.81
2 locust_77@test.com	9.09
3 locust_5@test.com	29.19
4 locust_6@test.com	40.28
5 locust_95@test.com	43.00
6 locust_40@test.com	54.17
7 locust_72@test.com	61.97
8 locust_100@test.com	66.35
9 locust_86@test.com	67.45
10 locust_3@test.com	73.74

2. إثبات نجاح النظام في منع الإنفاق المزدوج، حيث لم يظهر أي رصيد سالب (Negative Balance)، مما يؤكد أن المستخدم لم يتمكن من إنفاق مبلغ يتجاوز الرصيد المتاح في محفظته، رغم وجود عدد كبير من الطلبات المتزامنة.

Query Query History	
1 SELECT COUNT(*) AS negative_wallets FROM users_user WHERE wallet_balance < 0;	
Data Output Messages Notifications	
negative_wallets bigint	
1 0	

بعد: `POST /api/orders/checkout-wallet-async/`

تُعالج عمليات الدفع باستخدام المحفظة الإلكترونية بصورة غير متزامنة (Asynchronously) بهدف تحسين زمن الاستجابة وتقليل الوقت الذي ينتظره المستخدم. فعند استلام طلب الدفع، تقوم نقطة النهاية (Endpoint) بإرجاع الاستجابة HTTP 202 Accepted مباشرةً، مع وضع المهمة `process_wallet_payment_async` في قائمة الانتظار (Queue) ليتم تنفيذها في الخلفية.

تُنفَّذ عملية الدفع الفعلية داخل الدالة `checkout_with_wallet()` ضمن معاملة قاعدة بيانات ذرية (Atomic Transaction) باستخدام `transaction.atomic()`. وخلال هذه المعاملة يتم تطبيق آليات قفل تضمن سلامة البيانات، حيث يتم قفل سجل المستخدم باستخدام `select_for_update()` قبل التحقق من رصيد المحفظة وخصم المبلغ المطلوب، كما يتم قفل سجلات المنتجات قبل تعديل كميات المخزون.

تضمن هذه الآلية عدم تمكن الطلبات المتزامنة من استخدام الرصيد نفسه أو تعديل المخزون نفسه في الوقت ذاته، مما يمنع حدوث تعارضات ناتجة عن الوصول المتزامن إلى البيانات المشتركة.

النتيجة الآمنة (Safe Result): تتج أول عملية دفع صالحة تستوفي شروط الرصيد والمخزون، بينما يتم رفض أي محاولات مكررة أو متزامنة لاحقة بصورة آمنة إذا أصبح الرصيد أو المخزون غير كافٍ بعد تنفيذ العملية الأولى.

وبذلك تمنع آلية المزامنة مشكلة (Double Spending) وتحافظ على اتساق وسلامة بيانات المحفظة الإلكترونية والمخزون وسجلات الدفع، حتى في حالات الأحمال المرتفعة والتنفيذ المتزامن للطلبات.

 LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

4.74

Failures

11%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8500	1058.13	64	8598	421.08	1	0

3.6 الحالة 3: معالجة الدفع المزدوج

تحدث هذه الحالة عند إرسال طلب معالجة دفع لنفس الفاتورة أكثر من مرة

قبل الحل: إذا لم يتم قفل سجل الدفع، فقد يرى طلبان مختلفان حالة الدفع على أنها "قيد الانتظار" (Pending).

النتيجة المحتملة: قد تتم معالجة نفس الدفع مرتين.

في النظام الحالي، لا يتم إنشاء queue دفع جديد لكل محاولة، ولكن الخطر يكمن في ظهور unsafe endpoint بمحاولة الدفع الثانية وتعديل أو استبدال Transaction ID.

قبل: POST /api/orders/<id>/process-payment-unsafe/

تقرأ unsafe endpoint الطلب والدفع دون استخدام أقفال على مستوى الصفوف (Row Locks). ثم تتوقف لمدة 100 ملي ثانية، وبعدها تقوم بتحديد الدفع على أنه "مكتمل" (Completed) حتى لو كان هناك طلب آخر قد أكمله بالفعل. هذا يحاكي خطأ الدفع المزدوج.

تم إرسال طلبات دفع متكررة لنفس الطلب باستخدام endpoints غير آمنة. إن عدد الإخفاقات في أداة Locust لا يشير إلى فشل في الخادم، بل يدل على أن أداة Locust اكتشفت أن الدفع المكرر قد تم قبوله، مما يشكل تعارضاً في حالة السباق (Race-condition). (conflict)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	70	0	24000	31000	34000	22836.29	4100	33526	52	1.4	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	73.3	361	280	820	4200	479.99	131	23672	153.63	41.7	20.4
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	269	234	150	3400	3800	634.12	11	4094	190.96	16	13.5
POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	468	230	240	600	1300	310.91	117	10244	214.51	25.9	13

تثبت هذه الرسالة أنه تم دفع نفس الطلب أكثر من مرة عبر endpoint غير الآمنة. فقد سجلت أداة Locust هذا على أنه تعارض، لأن تكرار الدفع لنفس الطلب يُعد خطأ جسيماً.

1	POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	CatchResponseError('409 Conflict: duplicate payment processed for order=40722')
---	------	---	---

قاعدة البيانات: يبدو أن بعض المدفوعات قد اكتملت. بما أن النظام يستخدم علاقة One-to-One بين الطلب والدفع، فلن يظهر صف دفع ثاني لنفس الطلب. لذلك، لسنا بصدد البحث عن صفين، بل نبحث عما إذا تم تحديث نفس صف الدفع مرة أخرى.

Query

Query History

1

SELECT id, order_id, status, method, transaction_id, updated_at FROM orders_payment ORDER BY id DESC LIMIT 15;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 15

Page No: 1 of 1

	id [PK] bigint	order_id bigint	status character varying (20)	method character varying (20)	transaction_id character varying (255)	updated_at timestamp with time zone
1	41385	41385	COMPLETED	WALLET		2026-05-17 21:14:29.354126+00
2	41384	41384	COMPLETED	WALLET		2026-05-17 21:14:28.807648+00
3	41383	41383	COMPLETED	WALLET		2026-05-17 21:14:28.765957+00
4	41382	41382	COMPLETED	WALLET		2026-05-17 21:14:27.438806+00
5	41381	41381	COMPLETED	WALLET		2026-05-17 21:14:27.734512+00
6	41380	41321	PENDING	CREDIT_CARD		2026-05-17 21:14:27.460512+00
7	41379	41380	COMPLETED	WALLET		2026-05-17 21:14:27.452589+00
8	41378	41379	COMPLETED	WALLET		2026-05-17 21:14:26.790434+00
9	41377	41378	PENDING	CREDIT_CARD		2026-05-17 21:14:26.691908+00
10	41376	41377	PENDING	CREDIT_CARD		2026-05-17 21:14:26.623894+00
11	41375	41376	PENDING	CREDIT_CARD		2026-05-17 21:14:26.608075+00
12	41374	41375	PENDING	CREDIT_CARD		2026-05-17 21:14:26.605827+00
13	41373	41374	PENDING	CREDIT_CARD		2026-05-17 21:14:26.603794+00
14	41372	41373	PENDING	CREDIT_CARD		2026-05-17 21:14:26.572491+00
15	41371	41372	PENDING	CREDIT_CARD		2026-05-17 21:14:26.569813+00

توضح الصورة حالة قاعدة البيانات أثناء الاختبار الثالث للمتطلب الأول، حيث يتم فحص أوامر الشراء المرتبطة بمعاملات الدفع. يظهر أن بعض الطلبات قد وصلت إلى حالة الدفع "مكتمل" (Completed) بينما لا تزال أخرى في حالة "قيد الانتظار" (Pending)، مما يساعد على إثبات حدوث مشكلة في المعالجة المتزامنة قبل تطبيق آليات الحماية، مثل احتمال تنفيذ الدفع أو محاولة تحديث حالة الدفع بشكل غير متزامن.

Query

Query History

1 ; AS payment_status, p.transaction_id FROM orders_order o JOIN orders_payment p ON p.order_id = o.order_id

Data Output

Messages

Notifications

Showing rows: 1 to 15

Page 1 of 1

	order_id bigint	order_status character varying (20)	payment_status character varying (20)	transaction_id character varying (255)
1	41385	CONFIRMED	COMPLETED	
2	41384	CONFIRMED	COMPLETED	
3	41383	CONFIRMED	COMPLETED	
4	41382	CONFIRMED	COMPLETED	
5	41381	CONFIRMED	COMPLETED	
6	41380	CONFIRMED	COMPLETED	
7	41379	CONFIRMED	COMPLETED	
8	41378	CONFIRMED	PENDING	
9	41377	CONFIRMED	PENDING	
10	41376	CONFIRMED	PENDING	
11	41375	CONFIRMED	PENDING	
12	41374	CONFIRMED	PENDING	
13	41373	CONFIRMED	PENDING	
14	41372	CONFIRMED	PENDING	
15	41371	CONFIRMED	PENDING	

بعد: `POST /api/orders/<id>/process-payment/`

تضمن خدمة `process_payment()` معالجة الدفع مرة واحدة فقط من خلال تغليف العملية داخل `transaction.atomic()` وقفل كل من سجل الطلب (`Order`) وسجل الدفع المرتبط به (`Payment`) باستخدام `.select_for_update()`.

يمنع هذا الطلبات المتزامنة المتعددة من تعديل بيانات الدفع نفسها في نفس الوقت. قبل إكمال الدفع، تتحقق الخدمة من حالة الطلب وترفض الطلبات الملغاة. كما تتحقق مما إذا كان الدفع محدداً بالفعل كـ "مكتمل" (`COMPLETED`).

إذا تم اكتشاف محاولة دفع مكررة، يرفض النظام ذلك ويُرجع استجابة واضحة بدلاً من معالجة الدفع مرة أخرى.

النتيجة الآمنة: ينجح طلب الدفع الصحيح الأول فقط، بينما يتم رفض أي محاولة دفع مكررة أو متزامنة بأمان، مما يمنع معالجة الدفع المزدوج ويحافظ على اتساق حالة الطلب/الدفع.

هذا من واجهة مستخدم أداة Locust:

 LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

4.74

Failures

11%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	28	0	240	4700	35000	1762.95	47	35310	79.5	1.1	0

النتيجة من سطر الأوامر (`Terminal`) الخاص بنا

```
CASE 3 - Double Payment Processing
BEFORE endpoint : POST /api/orders/<id>/process-payment-unsafe/ twice
AFTER endpoint  : POST /api/orders/<id>/process-payment/ twice after safe order
Scenario : same customer tries to pay the same order twice - order+payment rows locked
First payment ok (200)           : 14
Duplicate blocked (400)          : 14
✓ PROTECTED - double payment prevented by pessimistic lock
```

3.7 الحالة 4: إلغاء الطلب أثناء معالجة الدفع

تحدث هذه الحالة عند إرسال طلب إلغاء في نفس الوقت الذي تتم فيه معالجة الدفع لنفس الطلب.

قبل الحل: يحاول كل من طلب الإلغاء وطلب معالجة الدفع تعديل نفس الطلب بشكل متزامن.

النتيجة المحتملة: قد ينتقل الطلب إلى حالة غير منطقية، مثل أن يتم إلغاؤه بعد أن اكتمل دفعه بالفعل.

قبل: `POST /api/orders/<id>/cancel-unsafe/`

لا تقوم endpoint غير الآمنة للإلغاء بقتل صف الطلب، ولا تفرض التزامن الصحيح لحالات الطلب (State Machine). نتيجة لذلك، يمكن للدفع والإلغاء تحديث نفس الطلب بشكل مستقل، مما ينتج عنه حالة غير صالحة يُلغى فيها الطلب بعد اكتمال الدفع.

يؤكد معدل الإخفاق المرتفع في أداة Locust وجود حالة race condition هذه، ويثبت أن القفل على مستوى الصفوف (Row-level Locking) مع التحقق من الحالة (State Validation) مطلوب لمنع إلغاء الطلبات المدفوعة بشكل خاطئ.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	80	0	27000	36000	37000	26431.5	5766	37496	52	0	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	2722	1374	360	840	3200	485.11	124	23070	154.32	51	27.9
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	880	809	190	3300	3600	479.38	27	6437	162.59	19.3	18.8
POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	1779	888	310	590	1100	344.28	128	4851	214.5	30.8	15.3
POST	Req1 BEFORE Case 4 cancel while payment processing POST /api/orders/{id}/cancel-unsafe/	442	441	290	510	890	314.19	129	1709	186	8.3	8.3

يشير الخطأ `Conflict: paid order was cancelled 409` إلى أن الدفع قد اكتمل، ولكن ال endpoint غير الآمنة قامت بإلغاء الطلب على أي حال. يؤكد هذا أن العمليات المتزامنة يمكن أن تُنتج بيانات غير متسقة قبل تطبيق الحل الآمن.

POST Req1 BEFORE Case 4 cancel while payment processing | POST /api/orders/{id}/cancel-unsafe/ CatchResponseError(409 Conflict: paid order was cancelled order=43248)

هذا هو سطر الأوامر (Terminal) عند انتهاء الاختبار:

يحسب سطر الأوامر الطلبات المرفوضة (المحمية)، بينما تحسب أداة Locust الإخفاقات غير المتوقعة. في هذه الحالة، يُعد رمز الحالة HTTP 400 أمراً متوقعاً لأن endpoint المحمية يجب أن ترفض محاولات الإلغاء غير الصالحة. لذلك، يُظهر سطر الأوامر الطلبات المرفوضة، ولكن أداة Locust تُبلغ عن صفر إخفاقات لأن النظام تصرف بشكل صحيح.

```
CASE 4 – Cancel Order While Payment Is Processing
BEFORE endpoint : POST /api/orders/<id>/cancel-unsafe/ after payment attempt
AFTER endpoint : POST /api/orders/<id>/cancel/ after safe order/payment attempt
Scenario : cancel races with payment – order row lock and state machine enforced
Cancel ok (200) : 1
Blocked – wrong state (400) : 441
```

يُظهر مخرج سطر الأوامر هذا "المتطلب 1 - الحالة 4" بعد تطبيق الحل. تستخدم ال endpoint المحمية `/api/orders/<id>/cancel/` القفل على مستوى الصفوف والتحقق من الحالة.

على الرغم من أن سطر الأوامر يُظهر (400) Blocked - wrong state، إلا أن أداة Locust تُبلغ عن صفر إخفاقات لأن استجابة HTTP 400 هي الاستجابة الآمنة المتوقعة في هذا السيناريو. ترفض نقطة النهاية المحمية محاولات الإلغاء غير الصالحة بشكل صحيح، لذا تسجل أداة Locust هذه الاستجابات كسلوك ناجح وليس كإخفاق في الاختبار.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	28	0	240	4700	35000	1762.95	47	35310	79.5	1.1	0
POST	Req1 AFTER Case 4 cancel while payment processing POST /api/orders/{id}/cancel/	4	0	86	140	140	88.68	22	142	53	0.2	0
POST	Req1 AFTER Case 5 POST											

نجح إلغاء واحد فقط، بينما تم رفض 441 طلباً برمز HTTP 400، مما يؤكد أنه لم يعد بإمكان إلغاء الطلبات المدفوعة أو قيد المعالجة بشكل خاطئ أثناء نشاط الدفع المتزامن.

تدمج لقطة قاعدة البيانات أحدث الطلبات مع سجلات الدفع الخاصة بها لاكتشاف انتقالات الحالة غير المتسقة أثناء الإلغاء المتزامن ومعالجة الدفع. تساعد في التحقق مما إذا كان الطلب قد وصل إلى تركيبة غير صالحة، مثل اكتمال الدفع مع إلغاء غير صحيح للطلب.

Query History

```

8 FROM orders_order o
9 JOIN orders_payment p ON p.order_id = o.id
10 ORDER BY o.id DESC
11 LIMIT 20;
12

```

Data Output Messages Notifications

Showing rows: 1 to 20 Page No: 1

	order_id bigint	order_status character varying (20)	payment_status character varying (20)	method character varying (20)	transaction_id character varying (255)	updated_at timestamp with time zone
1	45712	CONFIRMED	COMPLETED	WALLET		2026-05-18 17:39:17.860718+...
2	45711	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.524587+...
3	45710	CONFIRMED	COMPLETED	WALLET		2026-05-18 17:39:16.51462+00
4	45709	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.340982+...
5	45708	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.332425+...
6	45707	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.290199+...
7	45706	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.286699+...
8	45705	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.274587+...
9	45704	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.262724+...
10	45703	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.255763+...
11	45702	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.253233+...
12	45701	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 17:39:16.251263+...

Total rows: 20 Query complete 00:00:00.117

بعد: [POST /api/orders/<id>/cancel/](#)

تقوم خدمة الإلغاء الآمنة `cancel_order()` بنقل صف الطلب باستخدام `select_for_update()`، ولا تسمح بالإلغاء إلا عندما يكون الطلب في حالة "قيد الانتظار" (PENDING) أو "مؤكد" (CONFIRMED). بمجرد أن تقوم معالجة الدفع بتغيير حالة الطلب إلى "قيد المعالجة" (PROCESSING)، يتم رفض الإلغاء.

يستخدم الحل القفل التشاؤمي (Pessimistic Locking) مع `transaction.atomic()` و `select_for_update()` لقفل صف الطلب أثناء كل من معالجة الدفع والإلغاء. كما يطبق التحقق من الحالة لضمان أن كل انتقال لحالة الطلب أمر صالح.

النتيجة الآمنة: إذا اكتمل الدفع أولاً، يتم حظر الإلغاء. وإذا حدث الإلغاء أولاً، يتم حظر معالجة الدفع. لذلك، لم يعد من الممكن أن يصبح الطلب في حالة "ملغي" (CANCELLED) و "مدفوع" (PAID) في نفس الوقت.

تعد استجابات HTTP 400 أمراً متوقعاً بعد الحل. فهي تشير إلى أن ال `endpoint` المحمية رفضت طلبات الإلغاء بشكل صحيح عندما لم تكن حالة الطلب تسمح بذلك، مما يمنع إلغاء طلب مدفوع.

CASE 4 – Cancel Order While Payment Is Processing

BEFORE endpoint : `POST /api/orders/<id>/cancel-unsafe/` after payment attempt
 AFTER endpoint : `POST /api/orders/<id>/cancel/` after safe order/payment attempt
 Scenario : cancel races with payment – order row lock and state machine enforced
 Cancel ok (200) : 0
 Blocked – wrong state (400) : 4

لا يُظهر أي طلب مدفوع على أنه ملغي، مما يؤكد أن نقطة النهاية المحمية تمنع حالات الطلب/الدفع غير الصالحة.

Query

Query History

5

p.method,

6

p.transaction_id,

7

o.updated_at

8

FROM orders_order o

9

JOIN orders_payment p ON p.order_id = o.id

10

ORDER BY o.id DESC

11

LIMIT 20;

Data Output

Messages

Notifications

</

3.8 الحالة 5: حجز المنتجات / الحجز الزائد (Over-Reservation)

تحدث هذه الحالة عندما يحاول عدة مستخدمين حجز نفس المنتج الذي يمتلك مخزوناً منخفضاً في نفس الوقت.

قيل الحل: تقوم نقطة النهاية غير الآمنة للحجز بإنشاء حجوزات للمنتجات دون قفل صف المنتج أو التحقق من الكمية المحجوزة الأخيرة بشكل (Atomically).

النتيجة المحتملة: يمكن أن تصبح إجمالي الكمية المحجوزة النشطة أكبر من المخزون الفعلي المتاح، مما يعني أن النظام قام بحجز المنتج بشكل زائد.

قيل: `POST /api/products/<id>/reserve-unsafe/`

ت حسب endpoint غير الآمنة الحجوزات النشطة دون استخدام أقفال، ثم تتوقف لمدة 100 مللي ثانية، وبعدها تنشئ سجل OrderLock جديد. يمكن لجميع المستخدمين المتزامنين رؤية نفس الكمية المحجوزة القديمة وإنشاء حجوزات تتجاوز مخزون المنتج.

إحصائيات أداة Locust للمتطلب 1 - الحالة 5 (قبل الحل). استقبلت نقطة النهاية غير الآمنة

`/api/products/{id}/reserve-unsafe/` عدد 357 طلباً وجميعها فشلت، مما يُظهر أن الاختبار كشف بنجاح عن حالة السباق المتعلقة بالحجز الزائد.

LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

23.85

Failures

67%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	80	60	60000	60000	60000	57046.18	19714	60202	138.25	0	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	973	630	170	750	1100	278.2	2	37943	154.88	32.8	20.2
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	201	188	81	3300	7500	520.77	18	9573	155.48	6.1	5.9
POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	446	222	210	730	1500	365.19	116	29719	214.5	15.6	7.9
POST	Req1 BEFORE Case 4 cancel while payment processing POST /api/orders/{id}/cancel-unsafe/	117	104	180	540	740	281.93	117	6130	185.78	4.7	4.2
POST	Req1 BEFORE Case 5 POST /api/products/{id}/reserve-unsafe/ (over-reservation)	357	357	170	690	1000	207.35	2	2605	175.89	12	12

تعد استجابات HTTP 400 أمراً متوقعاً بعد تطبيق الحل. فهي تشير إلى أن endpoint المحمية رفضت طلبات الحجز بشكل صحيح عندما كان المخزون غير كافٍ، بدلاً من السماح بالحجز الزائد.

```
CASE 5 – Product Reservation / Over-Reservation
BEFORE endpoint : POST /api/products/<id>/reserve-unsafe/ (no product row lock)
AFTER endpoint : POST /api/products/<id>/reserve/ (SELECT FOR UPDATE)
Scenario : concurrent reservations for same product – unsafe over-reserves, safe blocks
Reservation ok (201) : 3
Blocked – low stock (400) : 23
```

رسالة الفشل `OVER-RESERVED: reserved=114 stock=149` تُظهر أن طلبات الحجز المتزامنة تسببت في تجاوز الكمية المحجوزة النشطة لحد الحجز الآمن المتوقع. يؤكد هذا أن ال endpoint غير الآمنة سمحت بسلوك حجز غير متسق قبل استخدام القفل على مستوى الصفوف.

الأداء قبل row-level locking:

Req1 BEFORE Case 5 | POST

/api/products/{id}/reserve-unsafe/ (over-reservation)

CatchResponseError(OVER-RESERVED: reserved=114 stock=149')

تُظهر قاعدة البيانات حالة حجز غير صالحة قبل الحل: منتج واحد ذو مخزون منخفض يحتوي على مخزون سالب وحجوزات نشطة في نفس الوقت. يؤكد هذا أن طلبات الحجز غير الآمنة المتزامنة يمكن أن تُفسد المخزون دون وجود قفل على مستوى الصفوف.

The screenshot shows a database client interface with a query editor and a results table. The query is as follows:

```
5 COALESCE(SUM(l.quantity), 0) AS active_reserved
6 FROM products_product p
7 LEFT JOIN products_orderlock l
8 ON l.product_id = p.id
9 AND l.expires_at > NOW()
10 WHERE p.description LIKE '%RACE CONDITION TEST PRODUCT%'
11 GROUP BY p.id, p.name, p.stock
12 ORDER BY active_reserved DESC;
```

The results table displays the following data:

	product_id bigint	name character varying (255)	stock integer	active_reserved bigint
1	4578	Exclusive MacBook Pro M3 Max	-81	77
2	4576	Limited Edition PlayStation 5 Bun...	10	0
3	4577	Last-Chance iPhone 15 Pro 1TB	10	0
4	4579	Flash Sale Sony Camera Kit	10	0
5	4580	Rare Nintendo Switch Bundle	10	0

وهذا من سطر الأوامر (Terminal)

CASE 5 – Product Reservation / Over-Reservation

BEFORE endpoint : POST /api/products/<id>/reserve-unsafe/ (no product row lock)

AFTER endpoint : POST /api/products/<id>/reserve/ (SELECT FOR UPDATE)

Scenario : concurrent reservations for same product – unsafe over-reserves, safe blocks

Reservation ok (201) : 5

Over-reserved / blocked count : 294

بعد: POST /api/products/<id>/reserve/

تقوم ال endpoint الآمنة باستدعاء create_order_lock()، والذي يُغلف عملية الحجز داخل Transaction ويقوم بقفل صف المنتج باستخدام select_for_update(). ولا يقوم الطلب بإنشاء الحجز إلا في حال توفر مخزون كافٍ.

يستخدم الحل القفل التشاؤمي (Pessimistic Locking) مع transaction.atomic() و select_for_update() لقفل صف المنتج أثناء عملية الحجز. يقوم النظام بقفل المنتج أولاً، والتحقق من الكمية المتاحة، ثم إنشاء الحجز فقط في حال كان المخزون كافياً.

النتيجة الآمنة: لا تُنشأ الحجوزات إلا ضمن حد المخزون المتاح، ويتم رفض أي طلب حجز إضافي.

LOCUST

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

http://nginx:80

STOPPED

4.74

11%

NEW

RESET

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	28	0	240	4700	35000	1762.95	47	35310	79.5	1.1	0
POST	Req1 AFTER Case 4 cancel while payment processing POST /api/orders/{id}/cancel/	4	0	86	140	140	88.68	22	142	53	0.2	0
POST	Req1 AFTER Case 5 POST /api/products/{id}/reserve/ (over-reservation protected)	26	0	260	4200	4700	692.6	22	4734	50.96	1.6	0

تعد استجابات HTTP 400 أمراً متوقعاً بعد الحل. فهي تشير إلى أن ال endpoint المحمية رفضت طلبات الحجز بشكل صحيح عندما كان المخزون غير كافٍ، بدلاً من السماح بالحجز الزائد.

CASE 5 – Product Reservation / Over-Reservation

BEFORE endpoint : POST /api/products/<id>/reserve-unsafe/ (no product row lock)

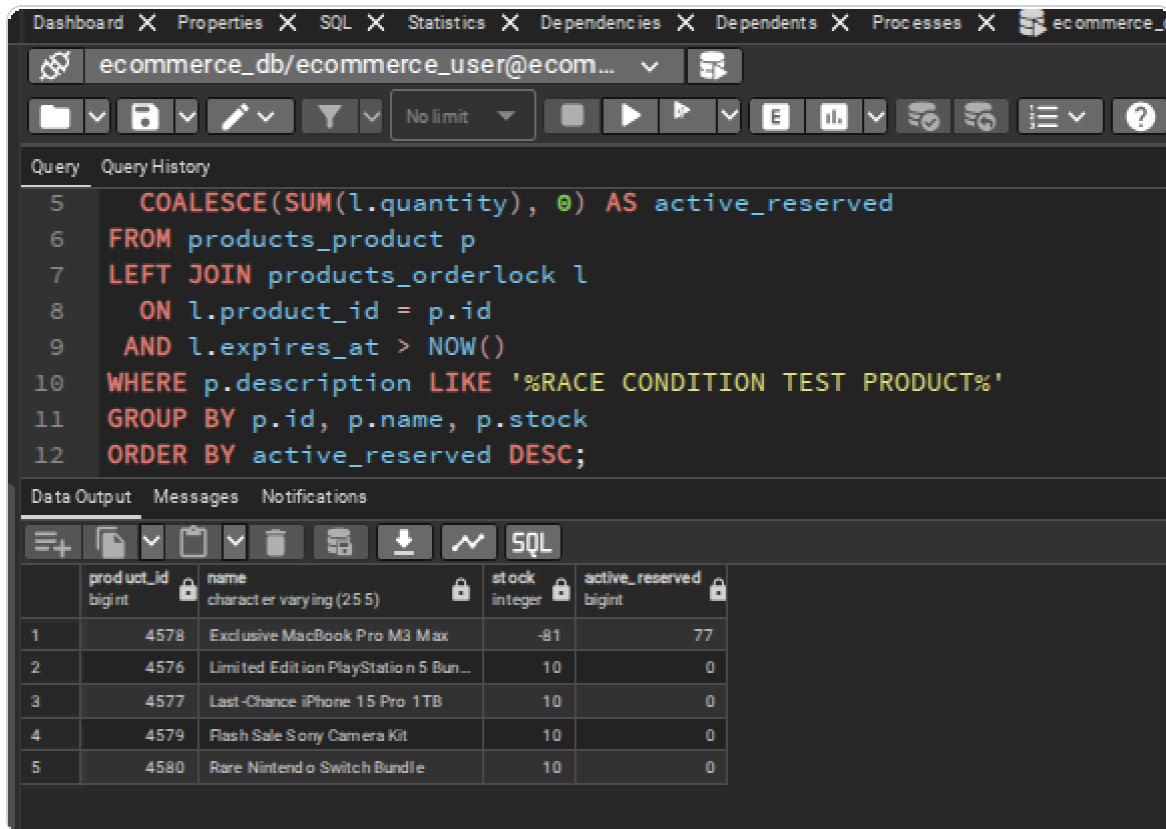
AFTER endpoint : POST /api/products/<id>/reserve/ (SELECT FOR UPDATE)

Scenario : concurrent reservations for same product – unsafe over-reserves, safe blocks

Reservation ok (201) : 3

Blocked – low stock (400) : 23

قاعدة البيانات: بعد الحل، ظلت جميع المنتجات المتأثرة بحالات السباق في حالة حجز "آمنة". أصبحت الكمية المحجوزة النشطة ضمن حدود المخزون المتاح، مما يؤكد أن القفل التشاؤمي يمنع الحجز الزائد.



4. المتطلب الثاني - إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

4.1 المشكلة

يجب على ال Backend التحكم في العمل المتوازي لكي لا تنهار تحت الضغط أو تصبح بطيئة بسبب عدم الاستغلال الأمثل للموارد. يوجد التزامن (Concurrency) على عدة مستويات: معالجة طلبات HTTP، واتصالات قاعدة البيانات، والعاملين في الخلفية (Background Workers)، وسلوك إعادة المحاولة (Retry Behavior).

4.2 ضوابط التحكم المُطبقة

الطبقة	آلية التحكم
HTTP server	فئة عامل (Worker Class) الخاصة بـ Gunicorn من نوع <code>gthread</code>
Container لكل تطبيق	<code>workers=8 -- threads=3 --</code>
السعة الإجمالية للتطبيق	<code>5 containers x 8 workers x 3 threads = 120 request handlers</code>

تجمع اتصالات (Pooling) من نوع Transaction في PgBouncer	ضغط اتصال قاعدة البيانات
<code>max_connections=200</code>	هامش PostgreSQL
<code>CONN_MAX_AGE=60</code>	إعادة استخدام اتصال Django
عاملون (Workers) مخصصون لكل queue وتزامن (Concurrency) ثابت	سعة Celery
تستخدم إعادة محاولة المخزون Optimistic تراجعاً زمنياً (Backoff) قصيراً	ضغط إعادة المحاولة

4.3 الإعدادات الحالية لـ Gunicorn

كل app container يبدأ بالأمر التالي:

```
gunicorn -- worker- class=gthread -- workers=8 -- threads=3 \
- - access- logfile=- -- error- logfile=- \
- - bind=0. 0. 0. 0: 8000 config. wsgi: application
```

حساب السعة:

```
Each app container: 8 workers x 3 threads = 24 request handlers
All app containers: 5 x 24 = 120 request handlers
```

4.4 التحكم في اتصالات قاعدة البيانات

يتصل كل من Django و Celery بـ PgBouncer عبر:

```
DB_HOST=pgbouncer
DB_PORT=5432
```

ثم يقوم PgBouncer بالاتصال بـ PostgreSQL على `db:5432`. هذا يحمي PostgreSQL من إنشاء اتصال مباشر لكل مسار طلب (Request Thread) أو مهمة Celery.


```
POOL_MODE=transaction
MAX_CLIENT_CONN=500
DEFAULT_POOL_SIZE=30
RESERVE_POOL_SIZE=10
```

4.5 التحكم في سعة Celery

الهدف	Concurrency	Queue	Worker
المهام العامة	2	celery	celery-worker
مهام البريد الإلكتروني والإشعارات	4	emails	celery_email_worker
مهام الدُفعات الثقيلة على قاعدة البيانات	1	batch	celery_batch_worker

يستخدم عامل الدُفعات (Batch Worker) تزامناً منخفضاً عمداً لأن تجميع الدُفعات (Batch Aggregation) يُنْهَك قاعدة البيانات بشكل كبير.

4.6 أوامر التحقق والاختبار

تشغيل اختبار السعة:

الحصول على الـ Token باستخدام بيانات الـ admin:

```
curl -X POST http://localhost/api/users/login/ \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@demo.com", "password": "admin123"}'
```

ثم:

```
docker compose run --rm \
  -e LOCUST_MODE=req2 \
  -e ADMIN_TOKEN=" <admin-token>" \
  locust -f locustfile.py --host http://nginx:80 \
  -u 500 --spawn-rate 50 --headless --run-time 3m
```

```
docker stats ecommerce_app1 ecommerce_app2 ecommerce_app3 ecommerce_app4
ecommerce_app5 \
ecommerce_pgouncer ecommerce_db ecommerce_nginx ecommerce_redis
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
6e3ad62108df	ecommerce_app1	51.74%	468MiB / 14.83GiB	3.08%	3.09MB / 3.57MB	24.6kB / 0B	32
161206695adc	ecommerce_app2	52.46%	451.9MiB / 14.83GiB	2.98%	3.08MB / 3.58MB	0B / 0B	27
e4b69b0f0cc1	ecommerce_app3	54.46%	467.9MiB / 14.83GiB	3.08%	3.06MB / 3.55MB	0B / 0B	30
1c83df10a54c	ecommerce_app4	55.67%	467.6MiB / 14.83GiB	3.08%	3.06MB / 3.56MB	393kB / 0B	28
87b7cf93e36f	ecommerce_app5	55.95%	468.3MiB / 14.83GiB	3.08%	3.05MB / 3.54MB	131kB / 0B	29
42709b451826	ecommerce_pgouncer	5.15%	5.926MiB / 14.83GiB	0.04%	15.4MB / 16.3MB	184kB / 8.19kB	1
9d2076f0b10a	ecommerce_db	13.79%	53.41MiB / 14.83GiB	0.35%	4.11MB / 5.03MB	13.4MB / 291kB	16
4fd155e7cc23	ecommerce_nginx	4.56%	27.07MiB / 14.83GiB	0.18%	10MB / 10.3MB	1.89MB / 4.1kB	25
5ccf4df61b8e	ecommerce_redis	0.24%	24.29MiB / 14.83GiB	0.16%	189kB / 243kB	57.9MB / 3.64MB	6

مراقبة اتصالات قاعدة البيانات النشطة:

```
docker exec ecommerce_db psql - U ecommerce_user - d ecommerce_db \
- c " SELECT count(*) , state FROM pg_stat_activity WHERE
datname=' ecommerce_db' GROUP BY state ORDER BY state; "
```

count	state
1	active
10	idle

(2 rows)

5. المتطلب 3: المعالجة غير المتزامنة (Asynchronous Queues)

5.1 المشكلة

هناك بعض العمليات التي لا ينبغي لها حجز أو إعاقة استجابة (HTTP Block the HTTP response). لذا، يُفضل معالجة رسائل البريد الإلكتروني الخاصة بالتأكيد، ورسائل الإلغاء، ومحاكاة مدفوعات المحفظة، وتنظيف الأقفال منتهية الصلاحية (Expired-lock cleanup)، والمهام المجمعة (Batch jobs) خارج مسار الطلب المباشر (Request path).

5.2 قبل التعديل: العمليات المتزامنة

تقوم `POST /api/orders/checkout-sync/` بإنشاء طلب ثم محاكاة إرسال البريد الإلكتروني عبر دالة التوقف المؤقت `sleep` قبل إرجاع HTTP 201.

وتقوم `POST /api/orders/checkout-wallet-sync/` بإجراء محاكاة لبوابة الدفع تستغرق 3 ثوانٍ كاملة داخل طلب HTTP نفسه. (عملية الدفع ← إنشاء الطلب ← الانتظار لإرسال البريد الإلكتروني داخل الطلب ← الاستجابة).

تُعد هذه النقاط مراجع مقارنة مفيدة لأن المستخدم يضطر فيها إلى انتظار انتهاء العمليات البطيئة.

5.3 بعد التعديل: Queues

تقوم `POST /api/orders/checkout/` بإنشاء الطلب، وإدراج مهمة `send_order_confirmation_email.delay(order.id)` في طابور المعالجة، ثم تُرجع HTTP 201 فوراً.

بينما تقوم `POST /api/orders/checkout-wallet-async/` بإدراج مهمة `process_wallet_payment_async.delay(user.id)` في طابور المعالجة، وتُرجع HTTP 202 فوراً.

يتم فصل توجيه المهام (Task routing) في ملف الإعدادات

المهمة	الطابور (Queue)
تأكيد الطلب عبر البريد الإلكتروني (Order confirmation email)	emails
إلغاء الطلب عبر البريد الإلكتروني (Order cancellation email)	emails
الدفعات المجمعة للمبيعات اليومية (Daily sales batch)	batch
تنظيف سلال التسوق المهجورة (Abandoned cart cleanup)	batch
تنظيف أقفال المنتجات منتهية الصلاحية (Expired product locks cleanup)	celery
إبطال التخزين المؤقت للمنتجات (Product cache invalidation)	celery

لقد قمنا بفصل (Queues) بناءً على الغرض منها، وذلك لضمان عدم انتهائنا بطابور واحد متكسد ومثقل بالمهام بشكل كبير.

5.4 أوامر التحقق من الصحة (Validation Commands)

مقارنة البريد الإلكتروني المتزامن (Synchronous email comparison):

```
docker logs -f ecommerce_celery_email_worker:
```

ال worker فارغ لأن رسائل البريد الإلكتروني يتم إرسالها داخل دورة حياة طلب HTTP نفسه، ولم يتم ترحيلها (Offloaded) إلى Celery.

```
- *** --- * ---
- ** ----- [config]
- ** ----- .> app:         config:0x7f03c5a4b5d0
- ** ----- .> transport:    redis://redis:6379/0
- ** ----- .> results:     redis://redis:6379/1
- *** --- * --- .> concurrency: 4 (prefork)
- - ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
- - ***** ---
- ----- [queues]
- .> emails             exchange=celery(direct) key=celery

[tasks]
. apps.cart.tasks.cleanup_abandoned_carts
. apps.core.tasks.cleanup_abandoned_carts
. apps.core.tasks.run_daily_sales_batch_naive_task
. apps.core.tasks.run_daily_sales_batch_task
. apps.orders.tasks.send_order_cancelled_email
. apps.orders.tasks.send_order_confirmation_email
. apps.orders.tasks.update_order_status_task
. apps.products.tasks.alert_low_stock
. apps.products.tasks.invalidate_product_cache
. apps.products.tasks.release_expired_locks_task
. apps.users.tasks.send_password_reset_email
. apps.users.tasks.send_welcome_email
. config.celery.debug_task

[2026-05-13 11:19:38,914: INFO/MainProcess] Connected to redis://redis:6379/0
[2026-05-13 11:19:38,918: INFO/MainProcess] mingle: searching for neighbors
[2026-05-13 11:19:39,929: INFO/MainProcess] mingle: all alone
[2026-05-13 11:19:39,942: INFO/MainProcess] email_worker@31f812c6db55 ready.
```

```
docker compose run --rm \
  -e LOCUST_MODE=req3_sync \
  locust -f locustfile.py --host http://nginx:80 \
  -u 5 --spawn-rate 1 --headless --run-time 30s
```

```

POST /api/users/login/ [sync-before] 5 0(0.00%) | 1187 615 2398 960 | 0.00 0.00
POST POST /api/orders/checkout-sync/ [REQ3 BEFORE - SLOW] 20 0(0.00%) | 2562 2019 4030 2019 |
0.90 0.00
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
Aggregated 71 0(0.00%) | 1048 4 4030 720 | 2.50 0.00

Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
POST /api/cart/add/ [sync-demo] 25 0(0.00%) | 16 9 27 16 | 0.80 0.00
DELETE /api/cart/clear/ [sync] 25 0(0.00%) | 737 4 1640 730 | 0.80 0.00
POST /api/users/login/ [sync-before] 5 0(0.00%) | 1187 615 2398 960 | 0.00 0.00
POST POST /api/orders/checkout-sync/ [REQ3 BEFORE - SLOW] 22 0(0.00%) | 2580 2019 4030 2019 |
0.90 0.00
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
Aggregated 77 0(0.00%) | 1059 4 4030 720 | 2.50 0.00

Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
POST /api/cart/add/ [sync-demo] 27 0(0.00%) | 16 9 27 16 | 0.80 0.00
DELETE /api/cart/clear/ [sync] 27 0(0.00%) | 722 4 1640 730 | 0.80 0.00
POST /api/users/login/ [sync-before] 5 0(0.00%) | 1187 615 2398 960 | 0.00 0.00
POST POST /api/orders/checkout-sync/ [REQ3 BEFORE - SLOW] 24 0(0.00%) | 2594 2019 4030 2019 |
0.90 0.00
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
Aggregated 83 0(0.00%) | 1062 4 4030 720 | 2.50 0.00

[2026-05-13 11:49:17,369] 55e5958f8fa3/INFO/locust.main: --run-time limit reached, shutting down

```

Mode: req3_sync

REQ 1 - Race Condition Safety

HOT product checkouts succeeded : 0
Stock-outs blocked by locking : 0

REQ 2 - Resource Management & Capacity Control

Total requests : 0
Errors : 0

Evidence to screenshot:

- Locust response time
- docker stats
- pg_stat_activity
- active_queues for Celery workers
- Docker Compose workers/concurrency
- Exponential backoff code in services.py

✓ No resource exhaustion detected

REQ 3 - Async Queues (BEFORE vs AFTER)

BEFORE (sync email): avg=2606ms samples=26

REQ 4 - Batch Processing

REQ 6 - Distributed Caching

[2026-05-13 11:49:17,417] 55e5958f8fa3/INFO/locust.main: Shutting down (exit code 0)

```

Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
POST /api/cart/add/ [sync-demo] 29 0(0.00%) | 16 9 27 16 | 0.80 0.00

```

POST /api/orders/checkout-sync/ [REQ3 BEFORE — SLOW]

Avg ≈ 2100ms

Min ≈ 2017ms

Max ≈ 2300ms

Fails = 0

: (Worker) عند مراقبة

```
- *** --- * ---
- ** ----- [config]
- ** ----- .> app:          config:0x7f03c5a4b5d0
- ** ----- .> transport:   redis://redis:6379/0
- ** ----- .> results:    redis://redis:6379/1
- *** --- * --- .> concurrency: 4 (prefork)
- ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
- ***** ---
- ----- [queues]
- .> emails          exchange=celery(direct) key=celery

[tasks]
. apps.cart.tasks.cleanup_abandoned_carts
. apps.core.tasks.cleanup_abandoned_carts
. apps.core.tasks.run_daily_sales_batch_naive_task
. apps.core.tasks.run_daily_sales_batch_task
. apps.orders.tasks.send_order_cancelled_email
. apps.orders.tasks.send_order_confirmation_email
. apps.orders.tasks.update_order_status_task
. apps.products.tasks.alert_low_stock
. apps.products.tasks.invalidate_product_cache
. apps.products.tasks.release_expired_locks_task
. apps.users.tasks.send_password_reset_email
. apps.users.tasks.send_welcome_email
. config.celery.debug_task

[2026-05-13 11:19:38,914: INFO/MainProcess] Connected to redis://redis:6379/0
[2026-05-13 11:19:38,918: INFO/MainProcess] mingle: searching for neighbors
[2026-05-13 11:19:39,929: INFO/MainProcess] mingle: all alone
[2026-05-13 11:19:39,942: INFO/MainProcess] email_worker@31f812c6db55 ready.
```

مقارنة البريد الإلكتروني غير المتزامن:

```
docker compose run --rm \
- e LOCUST_MODE=req3_async \
  locust -f locustfile.py --host http://nginx:80 \
- - users 5 --spawn-rate 1 --headless --run-time 30s
```

Aggregated		3	1(33.33%)	3655	15	5692	5300	0.12	0.12
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/users/login/ [async-after]	3	1(33.33%)	3655	15	5692	5300	0.12	0.12
Aggregated		3	1(33.33%)	3655	15	5692	5300	0.12	0.12
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/cart/add/ [async-demo]	4	0(0.00%)	95	68	119	89	0.00	0.00
DELETE	/api/cart/clear/ [async]	4	0(0.00%)	2556	50	5003	740	0.00	0.00
POST	/api/users/login/ [async-after]	5	1(20.00%)	5323	15	7971	5700	0.30	0.10
Aggregated		13	1(7.69%)	2863	15	7971	740	0.30	0.10
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/cart/add/ [async-demo]	7	0(0.00%)	74	32	119	69	0.00	0.00
DELETE	/api/cart/clear/ [async]	9	0(0.00%)	1160	16	5003	50	0.00	0.00
POST	/api/users/login/ [async-after]	5	1(20.00%)	5323	15	7971	5700	0.40	0.10
POST	POST /api/orders/checkout/ [REQ3 AFTER - FAST]	6	0(0.00%)	374	83	637	330	0.00	0.00
Aggregated		27	1(3.70%)	1474	15	7971	100	0.40	0.10
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/cart/add/ [async-demo]	17	0(0.00%)	80	32	196	67	0.50	0.00
DELETE	/api/cart/clear/ [async]	17	0(0.00%)	636	16	5003	49	0.50	0.00
POST	/api/users/login/ [async-after]	6	1(16.67%)	5993	15	9346	5700	0.40	0.00
POST	POST /api/orders/checkout/ [REQ3 AFTER - FAST]	15	0(0.00%)	249	69	637	180	0.40	0.00

REQUIREMENTS TEST REPORT

Mode: req3_async

REQ 1 – Race Condition Safety

HOT product checkouts succeeded : 0
 Stock-outs blocked by locking : 0

REQ 2 – Resource Management & Capacity Control

Total requests : 0
 Errors : 0

Evidence to screenshot:

- Locust response time
- docker stats
- pg_stat_activity
- active_queues for Celery workers
- Docker Compose workers/concurrency
- Exponential backoff code in services.py

✓ No resource exhaustion detected

REQ 3 – Async Queues (BEFORE vs AFTER)

AFTER (async Celery): avg=163ms samples=59

REQ 4 – Batch Processing

REQ 6 – Distributed Caching

REQ 3 – Async Queues (BEFORE vs AFTER)

AFTER (async Celery): avg=163ms samples=59

فحص قوائم الانتظار:

```
docker compose exec celery_worker celery - A config inspect
active_queues docker compose exec celery_email_worker celery - A
config inspect active_queues
docker compose exec celery_batch_worker celery - A config inspect
active_queues
```

```
[2026-05-13 11:19:39,929: INFO/MainProcess] mingie: all alone
[2026-05-13 11:19:39,942: INFO/MainProcess] email_worker@31f812c6db55 ready.
[2026-05-13 12:23:51,892: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[2b9e2d08-bc07-4281-8808-04b7fbae0a71] received
[2026-05-13 12:23:51,971: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[e8d79bd7-0748-4807-a895-520288ea340b] received
[2026-05-13 12:23:52,133: INFO/ForkPoolWorker-4] [EMAIL] Simulating confirmation email for Order #26922 to locust_60@test.com (sleep 2s) ...
[2026-05-13 12:23:52,159: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[1bb36504-1ddf-4ff2-a39f-da46f3397620] received
[2026-05-13 12:23:52,283: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[0c7eefb1-02d2-4711-acfc-4e31bd546881] received
[2026-05-13 12:23:52,300: INFO/ForkPoolWorker-2] [EMAIL] Simulating confirmation email for Order #26923 to locust_17@test.com (sleep 2s) ...
[2026-05-13 12:23:52,955: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[3559ca89-01df-4a13-8bec-e8f06a5434f6] received
[2026-05-13 12:23:53,512: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[2992d75d-c339-41fc-aa88-099426a4e9de] received
[2026-05-13 12:23:53,714: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[397b49e9-102e-40ae-9c90-7ba672e9b841] received
[2026-05-13 12:23:53,961: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[47b48b76-ebd6-49dc-aa4a-8cbabebd70e7] received
[2026-05-13 12:23:54,052: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[53d15954-26a9-42a0-a698-08cbaf82659c] received
[2026-05-13 12:23:54,134: INFO/ForkPoolWorker-4] [EMAIL] Confirmation email simulation complete for Order #26922
[2026-05-13 12:23:54,152: INFO/ForkPoolWorker-4] Task apps.orders.tasks.send_order_confirmation_email[2b9e2d08-bc07-4281-8808-04b7fbae0a71] succeeded
in 2.246826513999622s: None
[2026-05-13 12:23:54,302: INFO/ForkPoolWorker-2] [EMAIL] Confirmation email simulation complete for Order #26923
[2026-05-13 12:23:54,332: INFO/ForkPoolWorker-2] Task apps.orders.tasks.send_order_confirmation_email[e8d79bd7-0748-4807-a895-520288ea340b] succeeded
in 2.3369953300007182s: None
[2026-05-13 12:23:54,431: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[deb54d3c-4c6b-40c5-98aa-90d5a4b90ccc] received
[2026-05-13 12:23:54,568: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[91b9e62f-2323-477d-a76f-9efba8561246] received
[2026-05-13 12:23:54,585: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[9296a7e3-730c-46b3-bcb3-4c7b00397d6a] received
[2026-05-13 12:23:54,706: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[b14932bc-aeef-4dec-9dbb-dcfa211098c9] received
[2026-05-13 12:23:55,292: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[f9f86097-a533-45a7-a5ed-5330f27ad448] received
[2026-05-13 12:23:55,554: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[4e655fd8-3078-4b79-b040-a1bf465bb55e] received
[2026-05-13 12:23:55,831: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[965f6973-7601-479c-8d09-d1f16842630b] received
[2026-05-13 12:23:55,882: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[95662582-e792-4530-a669-74320e5ef562] received
[2026-05-13 12:23:56,129: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[bedb15dd-125d-4b40-ba70-9bbcd9ffeb15] received
[2026-05-13 12:23:56,015: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[58df7627-bc36-41dd-80b7-ffb79c564020] received
```

النتيجة المتوقعة:

HTTP response returns before the email/payment simulation finishes.
Celery worker logs show the background task processing later.

جدول المقارنة

Async queue	Sync	Measure
164ms	2594ms	Avg Response
69ms	2019ms	Min Response
637ms	4030ms	Max Response
works in Background	empty	Celery status
16x		speedup

6. المتطلب 4 - معالجة الدفعات (Batch Processing)

6.1 المشكلة

يمكن أن تشمل عملية تجميع المبيعات اليومية عدداً ضخماً من الطلبات (Orders). إن تحميل جميع الطلبات دفعة واحدة في الذاكرة أمر بسيط ومباشر، ولكنه غير قابل للتوسع (Does not scale)؛ لأن استهلاك الذاكرة يرتفع طردياً مع زيادة عدد الطلبات.

6.2 قبل التعديل: الدفعات العشوائية / غير المحسنة (Before: Naive Batch)

:(The demo endpoint)

```
POST /api/core/trigger- batch- naive/
```

الطوابير (queues)

run_daily_sales_batch_naive_task. تقوم هذه المهمة بتحميل جميع السجلات المتطابقة في ذاكرة بايثون (Python memory) ومعالجتها دفعة واحدة (In one pass).

أدلة السجلات المتوقعة (Expected log evidence):

```
[ BATCH- NAIVE] Loaded ALL X order objects into memory at once -- NO  
CHUNKING
```

6.3 بعد التعديل: الدفعات المجزأة (Chunked Batch)

:(The optimized endpoint)

```
POST /api/core/trigger- batch/
```

الطوابير (queues). تقوم المهمة بجمع معرفات الطلبات (Order IDs) المتطابقة ومعالجتها في مجموعات مجزأة ذات حجم ثابت (Fixed-size chunks). الثابت الافتراضي لبينة التشغيل النهائية (Production constant) هو:

```
CHUNK_SIZE = 50
```

```
[ BATCH] Chunk 1/N processed .  
 . . [ BATCH] Chunk 2/N  
processed . . . [ BATCH] Daily  
sales . . . COMPLETE
```

6.4 الجدولة والعزل (Scheduling and Isolation)

يقوم Celery Beat بجدولة تقرير المبيعات اليومي في تمام الساعة 1:00 صباحاً:

```
apps. core. tasks. run_daily_sales_batch_task
```

يتم توجيه المهمة إلى (Queue) المخصص، واستهلاكها بواسطة عامل (Worker) بمعدل تزامن قدره 1 (Concurrency 1). يضمن هذا الإجراء عزل عمليات قاعدة البيانات الثقيلة والخاصة بالدفعات (Batch database work) عن إشعارات البريد الإلكتروني والمهام الخلفية المعتادة الأخرى.

6.5 أوامر التحقق من الصحة (Validation Commands)

الدفعات العشوائية / غير المحسنة (Naive batch):

```
docker compose run -- rm \
- e LOCUST_MODE=req4_before \
- e ADMIN_TOKEN=" <admin- token>" \
locust - f locustfile. py -- host http: //nginx: 80 \
- - users 1 -- spawn- rate 1 -- headless -- run- time 30s
```

الدفعات:

```
docker compose run -- rm \
- e LOCUST_MODE=req4_after \
- e ADMIN_TOKEN=" <admin- token>" \
locust - f locustfile. py -- host http: //nginx: 80 \
- - users 1 -- spawn- rate 1 -- headless -- run- time 30s
```

مراقبة الخرج:

```
docker logs - f ecommerce_celery_batch_worker
```

```
[2026-05-18 11:48:50,815: INFO/MainProcess] task apps.core.tasks.run_daily_sales_batch_task[bf45de5b-d3c8-4e4c-9473-1798a226eb7e] received
[2026-05-18 11:48:50,816: INFO/ForkPoolWorker-1] [BATCH] Starting daily sales aggregation for 2026-05-17 (window 2026-05-17 00:00:00+00:00 -> 2026-05-18 00:00:00+00:00)
[2026-05-18 11:48:50,824: INFO/ForkPoolWorker-1] [BATCH] Chunk 1/348 processed: 10 orders, revenue=68645.50
[2026-05-18 11:48:50,824: INFO/ForkPoolWorker-1] [BATCH] Chunk 2/348 processed: 10 orders, revenue=42934.77
[2026-05-18 11:48:50,825: INFO/ForkPoolWorker-1] [BATCH] Chunk 3/348 processed: 10 orders, revenue=33993.43
[2026-05-18 11:48:50,825: INFO/ForkPoolWorker-1] [BATCH] Chunk 4/348 processed: 10 orders, revenue=7277.71
[2026-05-18 11:48:50,826: INFO/ForkPoolWorker-1] [BATCH] Chunk 5/348 processed: 10 orders, revenue=16383.72
[2026-05-18 11:48:50,826: INFO/ForkPoolWorker-1] [BATCH] Chunk 6/348 processed: 10 orders, revenue=12551.80
[2026-05-18 11:48:50,827: INFO/ForkPoolWorker-1] [BATCH] Chunk 7/348 processed: 10 orders, revenue=18995.29
[2026-05-18 11:48:50,827: INFO/ForkPoolWorker-1] [BATCH] Chunk 8/348 processed: 10 orders, revenue=9925.21
[2026-05-18 11:48:50,828: INFO/ForkPoolWorker-1] [BATCH] Chunk 9/348 processed: 10 orders, revenue=10015.93
[2026-05-18 11:48:50,828: INFO/ForkPoolWorker-1] [BATCH] Chunk 10/348 processed: 10 orders, revenue=15546.51
[2026-05-18 11:48:50,829: INFO/ForkPoolWorker-1] [BATCH] Chunk 11/348 processed: 10 orders, revenue=9493.87
[2026-05-18 11:48:50,829: INFO/ForkPoolWorker-1] [BATCH] Chunk 12/348 processed: 10 orders, revenue=10855.08
[2026-05-18 11:48:50,830: INFO/ForkPoolWorker-1] [BATCH] Chunk 13/348 processed: 10 orders, revenue=11599.58
[2026-05-18 11:48:50,830: INFO/ForkPoolWorker-1] [BATCH] Chunk 14/348 processed: 10 orders, revenue=10449.68
[2026-05-18 11:48:50,831: INFO/ForkPoolWorker-1] [BATCH] Chunk 15/348 processed: 10 orders, revenue=9385.87
[2026-05-18 11:48:50,831: INFO/ForkPoolWorker-1] [BATCH] Chunk 16/348 processed: 10 orders, revenue=10981.26
```

```
[2026-05-18 11:48:50,978: INFO/ForkPoolWorker-1] [BATCH] Chunk 338/348 processed: 10 orders, revenue=12015.42
[2026-05-18 11:48:50,979: INFO/ForkPoolWorker-1] [BATCH] Chunk 339/348 processed: 10 orders, revenue=11556.96
[2026-05-18 11:48:50,979: INFO/ForkPoolWorker-1] [BATCH] Chunk 340/348 processed: 10 orders, revenue=14609.48
[2026-05-18 11:48:50,980: INFO/ForkPoolWorker-1] [BATCH] Chunk 341/348 processed: 10 orders, revenue=11182.69
[2026-05-18 11:48:50,980: INFO/ForkPoolWorker-1] [BATCH] Chunk 342/348 processed: 10 orders, revenue=12218.84
[2026-05-18 11:48:50,981: INFO/ForkPoolWorker-1] [BATCH] Chunk 343/348 processed: 10 orders, revenue=14299.64
[2026-05-18 11:48:50,981: INFO/ForkPoolWorker-1] [BATCH] Chunk 344/348 processed: 10 orders, revenue=8832.82
[2026-05-18 11:48:50,982: INFO/ForkPoolWorker-1] [BATCH] Chunk 345/348 processed: 10 orders, revenue=13478.43
[2026-05-18 11:48:50,982: INFO/ForkPoolWorker-1] [BATCH] Chunk 346/348 processed: 10 orders, revenue=8897.18
[2026-05-18 11:48:50,983: INFO/ForkPoolWorker-1] [BATCH] Chunk 347/348 processed: 10 orders, revenue=8681.68
[2026-05-18 11:48:50,983: INFO/ForkPoolWorker-1] [BATCH] Chunk 348/348 processed: 7 orders, revenue=10335.39
[2026-05-18 11:48:50,984: INFO/ForkPoolWorker-1] [BATCH] Daily sales for 2026-05-17 COMPLETE - 3477 orders, total_revenue=3817931.64, processed in 348 chunk(s) of 10
[2026-05-18 11:48:50,984: INFO/ForkPoolWorker-1] Task apps.core.tasks.run_daily_sales_batch_task[bf45de5b-d3c8-4e4c-9473-1798a226eb7e] succeeded in 0.16793693699992218s:
venue': 3817931.64, 'chunks': 348, 'chunk_size': 10}
```

7. المتطلب 5 - توزيع الأحمال (Load Distribution)

7.1 المشكلة

يمكن أن يتحول مثل تطبيق Django/Gunicorn الواحد إلى عنق زجاجة (Bottleneck) عندما يقوم العديد من المستخدمين بالوصول إلى القسم الخلفي في نفس الوقت. يمتلك Gunicorn عدداً ثابتاً من العمال (Workers) وخيوط المعالجة (Threads)، لذا عندما ينشغل جميع المعالجين، تضطر الطلبات الجديدة إلى الانتظار.

كما أن وجود مثل تطبيق واحد يركز كل ضغط الطلبات على حاوية تطبيق واحدة، مما يخلق عنق زجاجة في الأداء ونقطة فشل واحدة (Point of failure) لطبقة HTTP.

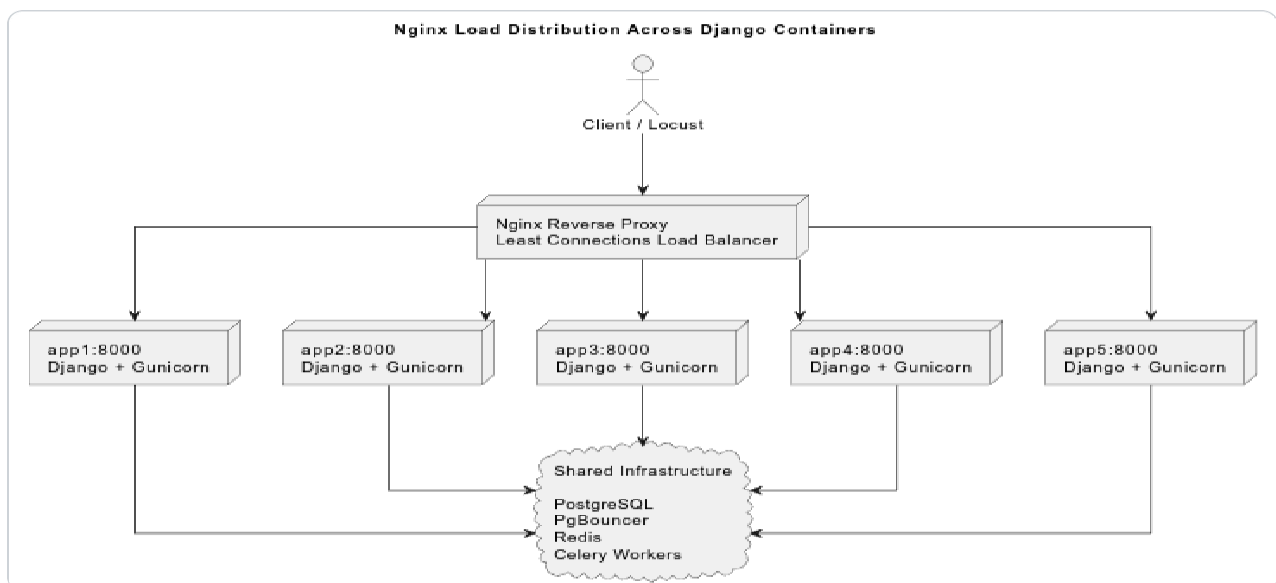
7.2 الحل: التوسع الأفقي باستخدام (Horizontal Scaling with)

يقوم المشروع بتشغيل خمس حاويات تطبيقات متطابقة من نوع Django/Gunicorn:

```
app1: 8000
app2: 8000
app3: 8000
app4: 8000
app5: 8000
```

تصل جميع حركات المرور الخارجية لبروتوكول HTTP أولاً إلى خادم Nginx، ومن ثم يقوم Nginx بتوجيه كل طلب إلى إحدى حاويات القسم الخلفي (Backend containers). تشترك جميع حاويات التطبيق في نفس البنية التحتية لكل من PostgreSQL و Redis و PgBouncer و Celery، وبالتالي فإن أي حاوية تطبيق قادرة على معالجة أي طلب وارد لمواجهة برمجة التطبيقات (API request).

يمنح هذا التصميم القسم الخلفي ميزة التوسع الأفقي (Horizontal scaling) على مستوى (Application layer):



7.3 استراتيجية Load balancing

خوارزمية موازنة الأحمال النشطة (Active load-balancing algorithm) هي الروابط الأقل (Least Connections):

```
least_conn;
```

تقوم خوارزمية (Least Connections) بإرسال كل طلب جديد إلى القسم الخلفي (Backend) الذي يحتوي على أقل عدد من الاتصالات النشطة في تلك اللحظة. ويُعد هذا الخيار أفضل من خوارزمية Round Robin البسيطة في هذا المشروع لأن طبيعة عبء العمل مختلطة (Mixed workload):

- يمكن أن تكون عمليات قراءة المنتجات سريعة، خاصة عند تخزينها مؤقتاً (Cached).
- عادة ما تكون عمليات سلة التسوق خفيفة (Light).
- قد يتطلب إدخال المنتجات إلى السلة عمليات كتابة في قاعدة البيانات.
- تُعد عملية الدفع (Checkout) أكثر ثقلًا لأنها تستخدم المعاملات (Transactions)، والتحقق من المخزون، وقفل السجلات على مستوى الصف (Row-level locking).

لا تعرف خوارزمية Round Robin ما إذا كان أحد الأقسام الخلفية مشغولاً بالفعل بطلب بطيء أم لا. في المقابل، تتكيف خوارزمية Least Connections مع النشاط الحالي للقسم الخلفي، مما يقلل من احتمالية تراكم طلبات الدفع الثقيلة على حاوية واحدة بينما تكون الحاويات الأخرى أقل انشغالاً.

تتضمن (Nginx upstream configuration) أيضاً آلية أساسية لمعالجة الأخطاء:

```
server app1: 8000 max_fails=3
fail_timeout=30s; server app2: 8000
max_fails=3 fail_timeout=30s; server app3:
8000 max_fails=3 fail_timeout=30s; server
app4: 8000 max_fails=3 fail_timeout=30s;
server app5: 8000 max_fails=3
fail_timeout=30s;
```

وهذا يعني أن Nginx يمكنه التوقف مؤقتاً عن إرسال حركة المرور إلى قسم خلفي يفشل بشكل متكرر.

7.4 القدرة الحالية

تقوم كل حاوية تطبيق بتشغيل:

```
8 Gunicorn workers
3 threads per worker
```

```
5 app containers x 8 Gunicorn workers x 3 threads = 120 request handlers
```

لا يعني هذا أن قاعدة البيانات يمكنها معالجة 120 عملية معاملات ثقيلة (Heavy transactional operations) في نفس الوقت

بأمان. لا يزال المتطلبات 2 (Requirement 2) يتحكم في الضغط الواقع على قاعدة البيانات من خلال PgBouncer، وإعادة استخدام اتصالات Django، وتحديد أعداد العاملين (Bounded worker counts). بينما يركز المتطلبات 5 (Requirement 5) على توزيع معالجة طلبات HTTP عبر حاويات تطبيقات متعددة.

7.5 لماذا لم يتم استخدام IP Hash

تقوم خوارزمية IP Hash بربط نفس عنوان الـ IP الخاص بالعميل بنفس القسم الخلفي (Backend). يُعد هذا مفيداً للجلسات الثابتة (Sticky sessions)، ولكن هذا المشروع لا يتطلب جلسات ثابتة لأننا نعتمد على التوثيق المستند إلى الرموز (Token-based authentication) كما أن واجهة برمجة التطبيقات (API) عديمة الحالة (Stateless).

ولم يتم اختيار خوارزمية IP Hash للأسباب التالية:

- لا تعتمد واجهة برمجة التطبيقات (API) على حالة الخادم المحلي (Local server state).
- أثناء اختبارات أداة Locust، قد تأتي العديد من الطلبات من نفس عنوان الـ IP أو نفس الحاوية.
- يمكن لخوارزمية IP Hash أن توجه حركة مرور ضخمة إلى قسم خلفي واحد (Backend)، مما يقلل من جودة وكفاءة توزيع الأحمال.

7.6 أوامر التحقق من الصحة (Validation Commands)

بدء تشغيل النظام بالكامل (Start the full system):

```
docker compose up -- build - d
```

تأكد من أن خادم Nginx وجميع حاويات التطبيق الخمسة تعمل بشكل صحيح:

```
docker  
ps
```

الحاويات المتوقعة (Expected containers):

```
ecommerce_nginx
ecommerce_app1
ecommerce_app2
ecommerce_app3
ecommerce_app4
ecommerce_app5
ecommerce_db
ecommerce_redis
ecommerce_pgouncer
```

تشغيل (Normal workload) عبر Nginx:

```
docker compose run -- rm \
- e LOCUST_MODE=normal \
- e ADMIN_TOKEN=" <admin- token>" \
locust -f locustfile.py --host http://nginx: 80 \
- - users 50 -- spawn- rate 10 -- headless -- run- time 1m
```

REQUIREMENTS TEST REPORT

Mode: normal

REQ 1 - Race Condition Safety (5 Cases)

[General] HOT checkouts via EcommerceUser succeeded : 0
[General] Stock-outs blocked : 0

REQ 2 - Resource Management & Capacity Control

Total requests : 1044

Errors : 8

Evidence to screenshot:

- Locust response time
- docker stats
- pg_stat_activity
- active queues for Celery workers
- Docker Compose workers/concurrency
- Exponential backoff code in services.py

✗ 8 unexpected errors

REQ 3 - Async Queues (BEFORE vs AFTER)

AFTER (async Celery): avg=17ms samples=294

REQ 4 - Batch Processing (BEFORE vs AFTER)

i Run with LOCUST_MODE=req4_before or req4_after to test batch processing

First hit (DB) : 35ms

Cache hit (Redis) : 7ms

Speedup : 4.7x ✓

[2026-05-18 14:53:52,871] 1eb7395f4d12/INFO/locust.main: Shutting down (exit code 1)

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/api/cart/ [VIEW]	94	0(0.00%)	5	2	167	3	1.57	0.00
POST	/api/cart/add/	564	8(1.42%)	9	2	246	6	9.42	0.13
DELETE	/api/cart/clear/	297	0(0.00%)	3	2	13	3	4.96	0.00
GET	/api/orders/ [LIST]	120	0(0.00%)	10	2	44	10	2.00	0.00
POST	/api/orders/checkout/ [ASYNC]	294	0(0.00%)	17	8	325	12	4.91	0.00
GET	/api/products/ [CACHE TEST]	1932	0(0.00%)	8	3	278	5	32.27	0.00
GET	/api/products/ [LIST]	512	0(0.00%)	6	3	228	5	8.55	0.00
GET	/api/products/[id]/	118	0(0.00%)	5	2	200	3	1.97	0.00
GET	/api/products/[id]/ [CACHE TEST]	455	0(0.00%)	4	2	170	3	7.60	0.00
POST	/api/users/login/	25	0(0.00%)	471	170	2031	200	0.42	0.00
POST	/api/users/login/ [browse]	25	0(0.00%)	390	163	1956	220	0.42	0.00
Aggregated		4436	8(0.18%)	12	2	2031	5	74.10	0.13

Response time percentiles (approximated)

Response time percentiles (approximated)												
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100% # reqs
GET	/api/cart/ [VIEW]	3	4	4	4	4	5	11	170	170	170	94
POST	/api/cart/add/	6	7	8	8	9	9	15	180	250	250	564
DELETE	/api/cart/clear/	3	4	4	4	4	5	6	7	13	13	297
GET	/api/orders/ [LIST]	10	12	12	13	15	17	30	35	45	45	120
POST	/api/orders/checkout/ [ASYNC]	12	12	13	14	29	49	76	100	330	330	294
GET	/api/products/ [CACHE TEST]	5	5	6	6	11	12	23	180	240	280	1932
GET	/api/products/ [LIST]	5	5	6	6	11	12	14	22	230	230	512
GET	/api/products/[id]/	3	4	4	4	4	5	12	200	200	200	118
GET	/api/products/[id]/ [CACHE TEST]	3	4	4	4	4	5	7	170	170	170	455
POST	/api/users/login/	200	340	360	1100	1500	1500	2000	2000	2000	2000	25
POST	/api/users/login/ [browse]	220	270	340	640	800	1100	2000	2000	2000	2000	25
Aggregated		5	6	6	9	12	13	87	200	1100	2000	4436
Error report												
# occurrences	Error											
8	POST /api/cart/add/: Cart add failed: 404											

كبدل آخر، قم بتشغيل Locust باستخدام واجهة المستخدم الرسومية للويب (Web UI):

```
docker compose run -- rm - p 8090: 8089 \
- e LOCUST_MODE=normal \
- e ADMIN_TOKEN=" <admin- token>" \
locust - f locustfile.py -- host http://nginx: 80
```

قيم واجهة مستخدم (Locust UI values):

```
Number of users: 50
Ramp up: 10
Host: http://nginx: 80
```

تخطيط عناوين الـ IP لحاويات التطبيق (Map app container IPs):

```
docker inspect - f ' {{. Name}} {{range . NetworkSettings. Networks}}
{{. IPAddress}}{{end}}' \
ecommerce_app1 ecommerce_app2 ecommerce_app3 ecommerce_app4
ecommerce_app5
```

```
[salah@Jarvis high-performance-e-commerce-backend]$ docker inspect -f '{{.Name}} {{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' \
ecommerce_app1 ecommerce_app2 ecommerce_app3 ecommerce_app4 ecommerce_app5
ecommerce_app1 172.18.0.10
ecommerce_app2 172.18.0.5
ecommerce_app3 172.18.0.12
ecommerce_app4 172.18.0.8
ecommerce_app5 172.18.0.11
```

التحقق من سجلات Nginx


```
docker logs ecommerce_nginx -- tail=100
```

الأدلة المتوقعة:

```
upstream=<app1- ip>: 8000
upstream=<app2- ip>: 8000
upstream=<app3- ip>: 8000
upstream=<app4- ip>: 8000
upstream=<app5- ip>: 8000
```

تؤكد هذه الأدلة المتوقعة أن Nginx يقوم بتوجيه الطلبات (Forwarding requests) إلى جميع حاويات القسم الخلفي الخمسة، بدلاً من إرسال حركة المرور (Traffic) بأكملها إلى (Instance) واحد من Django/Gunicorn.

[illegible]

```
docker logs -f ecommerce_app2 docker logs -f ecommerce_app3 docker logs -f ecommerce_app4 docker logs -f ecommerce_app5
```

تم استيفاء المتطلبات 5 (Requirement 5) لأن النظام يقوم بتوزيع الطلبات عبر خمسة حاويات لـ Django/Gunicorn باستخدام Nginx مع خوارزمية "الروابط الأقل" (Least Connections). تُظهر سجلات المنبع لـ Nginx (Nginx upstream logs) وصول الطلبات إلى جميع حاويات القسم الخلفي، مما يثبت التوزيع الأفقي لحركة المرور (Horizontal traffic distribution).

8. المتطلب 6 - استراتيجية التخزين المؤقت (Distributed Caching)

8.1 المشكلة

المشكلة الأساسية التي يعالجها هذا المتطلب هي الضغط العالي على قاعدة البيانات الناتج عن تكرار عمليات القراءة للبيانات نفسها، وما يترتب عليه من بطء في الاستجابة وانخفاض القدرة الاستيعابية للنظام.

أما الحل فهو استخدام Redis كذاكرة تخزين مؤقت موزعة لتخزين نتائج القراءة المتكررة وإعادة استخدامها بدلاً من تنفيذ الاستعلامات نفسها على قاعدة البيانات في كل مرة.

8.2 شرح اختبار Redis Distributed Caching

الهدف من هذا الاختبار هو إثبات تأثير استخدام Redis Distributed Cache على أداء endpoints الخاصة بالمنتجات في مشروع Django

قمنا بإجراء اختبارين

الأول هو اختبار BEFORE، ويمثل حالة النظام قبل الاستفادة من الكاش، حيث يتم إجبار النظام على قراءة البيانات من قاعدة البيانات مباشرة.

الثاني هو اختبار AFTER، ويمثل حالة النظام بعد تفعيل Redis Cache، حيث يتم تخزين نتائج القراءة الثقيلة في Redis ثم إعادة استخدامها بدل تنفيذ نفس الاستعلامات مرة أخرى على قاعدة البيانات.

الغاية الأساسية من المقارنة هي قياس الفرق في زمن الاستجابة، وعدد الطلبات التي يستطيع النظام معالجتها، ونسبة الفشل.

8.3 اختبار BEFORE (بدون كاش - يقرأ من DB مباشرة)

```
docker compose run --rm -p 8089:8089
```

```
e LOCUST_MODE=req6 before-
```

```
locust -f locust_tests/req6/locust_req6_cache_evidence.py --host http://nginx:80
```



Host

http://nginx:80

Status

CLEANUP

RPS

44.71

Failures

0%

EDIT

LOADING

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/products/ [REQ6 BEFORE - PRODUCT LIST DB READ]	1568	3	1900	4600	5900	2109.58	3	7752	376422.72	15.7	0
GET	/api/products/[id]/ [REQ6 BEFORE - PRODUCT DETAIL DB READ]	1364	4	130	1300	2500	341.42	5	4592	309.21	13.3	0.1
GET	/api/products/[id]/rating-summary/ [REQ6 BEFORE - RATING DB AGGREGATION]	983	3	140	1500	2300	360.69	3	3366	58.39	10.5	0
GET	/api/products/top-selling/ [REQ6 BEFORE - TOP SELLING DB AGGREGATION]	975	2	300	2000	3700	576.47	4	4954	3216.71	11.1	0
Aggregated		4890	12	380	3400	5000	959.13	3	7752	121440.96	50.6	0.1

يمثل هذا الاختبار حالة النظام قبل تفعيل Redis Cache، حيث يتم توجيه كل طلب وارد على endpoints المنتجات مباشرة إلى قاعدة البيانات دون أي وسيط تخزين مؤقت في هذه الحالة، كل request حتى لو كان مكرراً لنفس البيانات تماماً يضطر النظام لتنفيذ الاستعلام من جديد على قاعدة البيانات، بما يشمل عمليات الـ joins والـ aggregations الخاصة بكل endpoint (مثل حساب ملخص التقييمات أو ترتيب الأكثر مبيعاً). هذا النمط يحمل قاعدة البيانات بشكل مباشر ومتكرر، وكل ما زاد عدد المستخدمين المتزامنين (Concurrent Users)، زاد الضغط على القاعدة وزاد زمن الاستجابة، لأن كل طلب يدخل في طابور الاستعلامات بدون أي تخفيف النتائج المتوقعة في هذه الحالة هي: زمن استجابة أعلى (Median/Average/95%ile)، وقدرة أقل على معالجة عدد كبير من الطلبات في الثانية (RPS)، واحتمالية أعلى لظهور أخطاء أو تأخير ملحوظ تحت الضغط العالي.

8.4 اختبار AFTER (مع Redis Cache)

`docker compose run --rm -p 8089:8089`

`e LOCUST_MODE=req6_after locust -f locust_tests/req6/locust_req6_cache_evidence.py --host-http://nginx:80`

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/products/ [REQ6 AFTER PRODUCT LIST]	5586	0	38	390	1400	111.14	12	6858	272602	49.8	0
GET	/api/products/[id]/ [REQ6 AFTER - PRODUCT DETAIL]	4628	0	15	220	1200	68.99	4	4905	296.90	42.1	0
GET	/api/products/[id]/rating-summary/ [REQ6 AFTER - RATING SUMMARY]	3423	0	16	270	1300	75.83	4	1691	59	34.3	0
GET	/api/products/top-selling/ [REQ6 AFTER - TOP SELLING]	3422	0	15	270	1300	75.58	4	3633	3049	33.9	0
Aggregated		17059	0	23	310	1300	85.49	4	6858	89935.3	160.1	0

يمثل هذا الاختبار حالة النظام بعد تفعيل Redis Distributed Cache على نفس endpoints المنتجات. يعتمد النظام هنا على نمط Cache-Aside، حيث يقوم التطبيق أولاً بفحص Redis باستخدام cache key محدد، فإذا كانت البيانات موجودة يتم إرجاعها مباشرة من الذاكرة، أما إذا لم تكن موجودة يتم تنفيذ الاستعلام من قاعدة البيانات، ثم تخزين النتيجة في Redis مع فترة صلاحية مناسبة TTL.

تم تصميم الكاش في هذا النظام بحيث يتم تخزين بيانات القراءة العامة فقط، مثل بيانات المنتج المعروضة للمستخدم وقائمة المنتجات، والأكثر مبيعاً، وملخص التقييمات، مع تجنب تخزين كمية المخزون stock داخل الكاش العام السبب أن المخزون قيمة حساسة وسريعة التغير أثناء الحجز والشراء، لذلك يجب التعامل معها مباشرة من قاعدة البيانات داخل معاملات آمنة بدل الاعتماد على نسخة مخزنة قد تصبح قديمة.

كما يتم حذف الكاش المتأثر عند تعديل بيانات المنتج من خلال دوال invalidation داخل النظام، مثل

`invalidate_product_read_caches`، حتى لا تبقى بيانات قديمة بعد تغييرات الأدمن. وتظهر فائدة Redis Cache بشكل أوضح في endpoints المحسوبة مثل `/api/products/top-selling/` و `/api/products/[id]/rating-summary/` لأنها تعتمد على aggregation أو حسابات متكررة، وبالتالي فإن تخزين نتيجتها يقلل تكرار العمل على قاعدة البيانات.

نتيجة لذلك، أصبحت الطلبات المتكررة لنفس البيانات تقرأ من Redis بدل إعادة تنفيذ نفس الاستعلامات على قاعدة البيانات، مما يقلل الضغط على قاعدة البيانات ويحسن زمن الاستجابة. كما أن وجود TTL مع آلية حذف الكاش عند التعديل يساعد على تحقيق توازن بين الأداء وسلامة البيانات.

النتائج الفعلية لهذا الاختبار أظهرت تحسناً واضحاً بلغ متوسط زمن الاستجابة الكلي 85.49، ووصل معدل المعالجة إلى 160.1 RPS،

مع نسبة فشل 0% ضمن إجمالي 17059 طلب، كذلك سجل //endpoint api/products متوسط 111.14ms و 95%ile عند 390ms، بينما كان api/products/[id] الأسرع بمتوسط 68.99ms. هذه النتائج تؤكد أن استخدام Redis Cache حسن استقرار النظام وأدائه، مع الحفاظ على سلامة بيانات المخزون بعدم تخزينها في الكاش العام.

9. المتطلب 7 - التحكم في الأقفال (Concurrency Control)

9.1 المشكلة

عندما تنتهي صلاحية ذاكرة التخزين المؤقت (Cache Expiration) أو يتم مسحها، وتتراكم عدة طلبات HTTP لاكتشاف أن الكاش فارغ، فإنها جميعاً ستقوم بتنفيذ استعلامات متطابقة لقاعدة البيانات لإعادة بناء نفس البيانات. هذه المشكلة تُعرف بـ "زلزلة الكاش" (Cache Stampede)، وهي تضغط بشكل كبير ومفاجئ على قاعدة البيانات بدلاً من السماح لطلب واحد فقط بإعادة بناء الكاش.

9.2 شرح نتيجة BEFORE (بدون أقفال موزعة)

في هذه الحالة، تم مسح ذاكرة التخزين المؤقت (Redis Cache) تماماً، ثم تم إرسال 100 طلب متزامن إلى كل نقطة نهاية (Endpoint) دون استخدام آلية Redis Distributed Lock.

كشفت النتائج عن مشكلة *Cache Stampede* بوضوح تام. عند وصول الطلبات بشكل متقارب، وجدت مجموعة من الطلبات أن الكاش فارغ في نفس اللحظة. وبدون وجود آلية تنسيق (Lock) بينها، قام كل طلب بشكل مستقل بإعادة بناء نفس بيانات الكاش من قاعدة البيانات:

- في `/api/products/top-selling/`: نفذ النظام 64 عملية إعادة بناء من قاعدة البيانات (DB Rebuilds) من أصل 100 طلب. هذا يعني أن 64 طلباً تعرضوا لحالة عدم وجود بيانات في الكاش (Cache Miss)، فقرأوا البيانات من قاعدة البيانات وأعادوا كتابة نفس الـ `cache key` بشكل مكرر.

- في `/api/products/`: حدث نفس السلوك تقريباً، حيث تم تنفيذ 65 عملية إعادة بناء (DB Rebuilds) من أصل 100 طلب.

REQ7 BEFORE RESULT - /api/products/top-selling/	
Metric	Value
Total Requests	100
Failures	0
Cache HIT	36
DB Rebuilds	64
Fallback DB Reads	0
Avg Wait Time (ms)	0
REQ7 BEFORE RESULT - /api/products/	
Metric	Value
Total Requests	100
Failures	0
Cache HIT	35
DB Rebuilds	65
Fallback DB Reads	0
Avg Wait Time (ms)	0

LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

23.5

Failures

0%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/products/	100	0	920	1100	1100	877.65	360	1069	91081	0	0
GET	/api/products/top-selling/	100	0	540	650	690	513.68	255	687	3323.8	0	0
	Aggregated	200	0	630	1000	1100	695.66	255	1069	47202.4	0	0

تثبت هذه الأرقام أن المشكلة لا تكمن في غياب الكاش بحد ذاته، بل في أن عدة طلبات تحاول تعويض هذا الغياب في نفس الوقت، مما يضاعف الضغط على قاعدة البيانات بدلاً من أن يقوم طلب واحد فقط ببناء الكاش.

دليل الأداء (Locust): لم تحدث أي أخطاء (Failures)، لكن زمن الاستجابة كان مرتفعاً بشكل ملحوظ. في `/api/products/` وصل المتوسط (Median) إلى 920 مللي ثانية، وبلغت النسبة المئوية 95% 1100 مللي ثانية. هذا السلوك منطقي تماماً؛ لأن عدداً كبيراً من الطلبات لم يقرأ من Redis (السرّيع)، بل قام بإعادة بناء البيانات من قاعدة البيانات (البطيئة). أما `/api/products/top-selling/` فكانت أخف حجماً، وبالتالي كانت أزمنتها أقل، لكنها عانت من نفس المشكلة المنطقية: 64 عملية إعادة بناء لنفس البيانات بدلاً من عملية واحدة.

الخلاصة: لا يكفي وجود Redis Cache وحده لحماية قاعدة البيانات عند لحظة فقدان الكاش. تحت الضغط المتزامن، تتحول حالات عدم وجود الكاش (Cache Misses) المتعددة إلى استعلامات مكررة ومكلفة لقاعدة البيانات، وهذا هو بالضبط السلوك الذي تعالجه آلية **Redis Distributed Lock** في حالة AFTER.

9.3 شرح نتيجة AFTER (باستخدام أقفال موزعة)

في هذه الحالة، تم تشغيل نفس الاختبار بنفس عدد الطلبات (100 طلب متزامن)، ولكن هذه المرة مع تفعيل Redis Distributed Lock لكل مفتاح كاش (Cache Key). الهدف هنا ليس منع كل الطلبات من رؤية الكاش فارغاً في البداية، بل منعها من تنفيذ إعادة بناء مكررة من قاعدة البيانات.

أظهرت النتائج نجاح آلية القفل بشكل مباشر وحاسم:

- في `/api/products/top-selling/`: انخفض عدد عمليات إعادة البناء (DB Rebuilds) من 64 في حالة BEFORE إلى 1 فقط. هذا يعني أن طلباً واحداً فقط هو من حصل على القفل (Acquired Lock)، وقام ببناء البيانات من قاعدة البيانات، ثم كتب النتيجة في Redis. في نفس الوقت، 40 طلباً أخرى لم تذهب إلى قاعدة البيانات إطلاقاً، بل انتظرت فترة قصيرة جداً، ثم قرأت النتيجة من Redis بمجرد أن أصبحت جاهزة (وهو ما يظهر بالتقرير كـ `Served After Wait = 40` و `Fallback` و `DB Reads = 0`).
- في `/api/products/`: ظهر نفس السلوك الممتاز؛ حيث انخفض عدد عمليات الإعادة من 65 إلى 1 فقط. بدلاً من أن تعيد بقية الطلبات بناء نفس البيانات، انتظر 63 طلباً ثم حصلوا على النتيجة من Redis دون تنفيذ أي استعلام مستقل لقاعدة البيانات. وجود علامة `Protected = YES` و `Fallback DB Reads = 0` يؤكد أن القفل نجح في حماية قاعدة البيانات ولم يضطر النظام للتراجع (Fallback) إلى الاستعلام المباشر.

REQ7 AFTER RESULT – /api/products/top-selling/	
Metric	Value
Total Requests	100
Failures	0
Cache HIT	58
DB Rebuilds	1
Served After Wait	40
Fallback DB Reads	0
Avg Wait Time (ms)	41
Protected	YES

REQ7 AFTER RESULT – /api/products/	
Metric	Value
Total Requests	100
Failures	0
Cache HIT	36
DB Rebuilds	1
Served After Wait	63
Fallback DB Reads	0
Avg Wait Time (ms)	155
Protected	YES

 LOCUST

Host
http://nginx:80

Status
CLEANUP

RPS
25.37

Failures
0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/products/	100	0	410	500	510	391.33	219	508	91191.17	0	0
GET	/api/products/top-selling/	100	0	490	550	570	483.11	378	569	3437.75	0	0
	Aggregated	200	0	460	550	560	437.22	219	569	47314.46	0	0

دليل الأداء (Locust): لم تسجل أداة Locust أي أخطاء (Failures)، وانخفض زمن الاستجابة في `/api/products/` بشكل واضح، حيث هبط المتوسط (Median) من 920 مللي ثانية إلى 410 مللي ثانية. أما `/api/products/top-selling/` فحافظت على استقرارها مع أداء أفضل.

الخلاصة: نجح Redis Distributed Lock في تحويل زلزلة الكاش (Cache Stampede) من عشرات عمليات الإعادة المكررة (DB Rebuilds) إلى عملية واحدة فقط، بينما اكتفت الطلبات الأخرى بالانتظار للحصول على نتيجة الكاش الجاهزة. بذلك، لا يمنع القفل حدوث حالة (Cache Miss) أولية، لكنه يمنع الأثر الخطير لها وهو تكرار نفس الاستعلام على قاعدة البيانات تحت الضغط المتزامن.

10. المتطلب 8 - سلامة المعاملات (Transaction Integrity / ACID)

10.1 تحديد حالات استخدام المعاملات

- إضافة عنصر للسلة (Add to Cart)
- تعديل كمية عنصر في السلة (Update Cart Item)
- حذف عنصر من السلة (Remove from Cart)
- حجز مؤقت للمنتج (Reserve Product)
- إنشاء الطلب (Create Order)
- إلغاء الطلب (Cancel Order)
- معالجة الدفع (Process Payment)
- الدفع بالمحفظة (Checkout with Wallet)
- إعادة تخزين (Admin - Restock Product)
- تحديث حالة الطلب (Admin - Update Order Status)

10.2 كيفية تطبيق المعاملات في كل حالة

إضافة عنصر للسلة

تم تطبيق المعاملات هنا مع Optimistic Locking ، وذلك لأن عملية الإضافة تقرأ البيانات أولاً ثم تعديلها، وهي قابلة للإعادة في حال حدوث تعارض. لا نريد قفل الصف المجرد إضافة عنصر للسلة لأن ذلك يعطل بقية المستخدمين دون مبرر

تضمن هذه الوحدة أنه إذا أرسل نفس المستخدم طلبين متزامنين على نفس عنصر السلة في آن واحد، ينفذ الأول ويُعيد الثاني محاولته تلقائياً، كما تضمن أنه لا يمكن لأي تغيير جزئي أن يحفظ في قاعدة البيانات إذا فشلت العملية في أي مرحلة.

يتم ذلك عبر عدة خطوات:

1. قراءة بيانات عنصر السلة ورقم إصداره الحالي من قاعدة البيانات دون قفل، والتحقق من توافر الكمية المطلوبة في المخزون.
2. محاولة تحديث الصف بشرط أن يكون رقم الإصدار لم يتغير. إذا نجح التحديث فذلك يعني أنه لم يتدخل طلب آخر، وإذا فشل فذلك يعني أن طلباً آخر عدل نفس عنصر السلة قبله، ويُعاد التنفيذ من البداية تلقائياً.
3. في حال لم يكن العنصر موجوداً في السلة بعد ، يتم إنشاؤه كإدخال جديد داخل نفس الوحدة. تعديل كمية عنصر في السلة

تعديل كمية عنصر في السلة

تم تطبيق المعاملات هنا مع Pessimistic Locking ، وذلك لأن تعيين الكمية إلى قيمة محددة يجب أن يفوز فيها آخر كاتب بوضوح تام دون أي غموض القفل المسبق يسلسل الوصول ويمنع التعارض.

تضمن هذه الوحدة أنه لا يمكن لطلبين متزامنين أن يُعدّلا نفس عنصر السلة في آن واحد. كما تضمن عدم حفظ أي تغيير جزئي إذا فشلت العملية.

يتم ذلك عبر عدة خطوات:

1. قفل صف عنصر السلة في قاعدة البيانات بشكل حصري، مما يمنع أي تعديل متزامن على نفس الصف حتى انتهاء العملية.
2. تحديث الكمية إلى القيمة الجديدة وزيادة رقم الإصدار. حفظ التغييرات وتحرير القفل تلقائياً عند اكتمال العملية.

حذف عنصر من السلة

تم تطبيق المعاملات هنا مع Pessimistic Locking، لأن عملية الحذف يجب أن تكون حصرية، ولا يجوز لأى عملية أخرى تعديل نفس الصف أثناء تنفيذها.

تضمن هذه الوحدة أن العنصر يحذف بشكل كامل أو لا يحذف على الإطلاق كما تضمن أن قاعدة البيانات تبقى نظيفة في حال حدوث أي خطأ أثناء العملية، مما يسمح بإعادة المحاولة بأمان.

يتم ذلك عبر عدة خطوات:

1. قفل صف عنصر السلة المراد حذفه بشكل حصري.
2. حذف الصف من قاعدة البيانات وتحرير القفل عند الاكتمال.

حجز مؤقت للمنتج

تم تطبيق المعاملات هنا مع Pessimistic Locking، لأن الحجز يعتمد على قراءة المخزون وإنشاء سجل الحجز كوحدة لا تتجزأ، ولا يجوز أن يتحقق مستخدمان من نفس المخزون ثم يحجزانه معاً دون تنسيق.

تضمن هذه الوحدة أن الحجز ينشأ فقط إذا كان المخزون متاحاً بالفعل، وأن لا حجزين متعارضين ينشآن في نفس اللحظة.

ملاحظة مهمة هذه العملية لا تعدل كمية المخزون الفعلية، بل تنشئ فقط سجل حجز مؤقتاً يثبت أن المستخدم قد حجز الكمية لمدة عشر دقائق الخصم الفعلي من المخزون يحدث لاحقاً عند إنشاء الطلب.

يتم ذلك عبر عدة خطوات:

1. قفل صف المنتج في قاعدة البيانات بشكل حصري.
2. التحقق من كفاية المخزون تحت القفل لضمان دقة البيانات المقروءة.
3. إنشاء سجل الحجز المؤقت المرتبط بالمستخدم والمنتج والكمية وتاريخ الانتهاء.

إنشاء الطلب

تم تطبيق المعاملات هنا في أكثر العمليات حساسية في المتجر الإلكتروني تضمن هذه الوحدة تنفيذ خصم المخزون وإنشاء الطلب وسجل الدفع معاً كوحدة واحدة لا تتجزأ، بحيث لا يمكن أن ينشأ طلب دون خصم مخزون، ولا أن يخصم مخزون دون إنشاء طلب.

بما أن هذه العملية تعدل جداول متعددة في نفس الوقت، تم استخدام Pessimistic Locking وترتيب المنتجات بترتيب ثابت لمنع حالة الانتظار المتبادل بين الطلبات المتزامنة.

يتم ذلك عبر عدة خطوات:

1. قراءة عناصر السلة وترتيب معرفات المنتجات تصاعدياً، ثم قفل صفوف المنتجات في قاعدة البيانات بهذا الترتيب الثابت. هذا يضمن أن أى طلبين متزامنين يقفلان المنتجات بنفس الترتيب دائماً، ويقضى على احتمال الانتظار المتبادل بينهما.
2. التحقق من كفاية المخزون لكل عنصر في السلة تحت القفل، مما يضمن أن البيانات المقروءة دقيقة ولم يُعَدَّلها أحد آخر.
3. خصم المخزون من كل منتج وتسجيل كل تغيير في سجل المخزون الأغراض التدقيق والمراجعة اللاحقة.
4. إنشاء الطلب وبنوده، ثم تأكيد حالة الطلب وإنشاء سجل الدفع بحالة معلقة ريثما يُؤكد الدفع في خطوة لاحقة.
5. مسح السلة وإبطال الكاش تحذف عناصر السلة وتلغي البيانات المخزنة مؤقتاً لضمان تحديث الصفحات الرئيسية فوراً.

إلغاء الطلب

تم تطبيق المعاملات هنا مع Pessimistic Locking لضمان أن عملية الإلغاء واسترداد حالة الطلب تحدثان كوحدة واحدة لا تتجزأ.

تضمن هذه الوحدة أن لا يلغى طلب إلا بعد التأكد من أنه في حالة تسمح بالإلغاء، وأن لا يتداخل إلغاءه على نفس الطلب في آن واحد.

يتم تطبيق هذا المنطق عبر عدة خطوات:

1. قفل صف الطلب في قاعدة البيانات بشكل حصري، ثم التحقق من حالته إذا كان الطلب في حالة لا تسمح بالإلغاء كأن يكون قد شحن بالفعل، يتم إيقاف العملية وإعادة رسالة ملائمة دون أي تعديل.
2. تعيين حالة الطلب إلى ملفى وحفظها في قاعدة البيانات.

معالجة الدفع

تم تطبيق المعاملات هنا مع Pessimistic Locking ، لمنع مشكلة الدفع المزدوج وضمان أن الدفع يعالج مرة واحدة فقط مهما تكررت المحاولات.

تضمن هذه الوحدة أنه لا يمكن لأي استخدام آخر التفاعل مع الطلب أو سجل الدفع أثناء إتمام العملية، وأنه في حال حدوث أى خطأ تلقى جميع التغييرات تلقائياً.

يتم ذلك بإتباع الخطوات التالية:

1. ترتيب الأقفال لمنع الجمود (Sequential Locking): يتم قفل صف الطلب أولاً ثم قفل صف الدفع وهذا الترتيب الثابت يمنع حالة الانتظار المتبادل إذا حاول نفس المستخدم الدفع مرتين في آن واحد.
2. منع الدفع المزدوج بمجرد قفل الطلب، يتم فحص حالة الدفع الحالية. إذا كانت الحالة مكتملة بالفعل ترفض العملية فوراً دون أي تعديل. هذا يحمي المستخدمين من الدفع مرتين إذا ضغطوا على زر الدفع مرتين في آن واحد.

تحديث سجل الدفع وحالة الطلب يتم تعيين حالة الدفع إلى مكتمل وحالة الطلب إلى قيد المعالجة كخطوة واحدة غير قابلة للتجزئة، مما يضمن أنهما دائماً متزامنان

الدفع بالمحفظة

تم تطبيق المعاملات هنا على ثلاثة مستويات متداخلة في آن واحد حماية رصيد المحفظة، وحماية مخزون المنتجات، وإنشاء الطلب الهدف الجوهري هو ضمان أن خصم المحفظة وخصم المخزون وإنشاء الطلب يحدثون معاً أو لا يحدث أي منها.

يتم ذلك بإتباع الخطوات التالية:

1. ترتيب العمليات لمنع الجمود (Deadlocks): تم الالتزام بترتيب دقيق وموحد عند الأقفال: المستخدم أولاً ثم المنتجات مرتبة بترتيب ثابت. هذا يضمن أن أي طلبين متزامنين يتبعان نفس مسار الأقفال ولا يدخلان في انتظار متبادل
2. سلامة العمليات المالية يتم التحقق من وجود رصيد كاف في محفظة المستخدم تحت القفل، مما يضمن أن لا يخصم رصيد غير موجود. يتم خصم المبلغ من رصيد المحفظة وإنشاء سجل الدفع من نوع محفظة داخل نفس الوحدة، مما يربط كل خصم بطلب محدد يمكن الرجوع إليه.
3. خصم المخزون والتحقق منه يخصم مخزون كل منتج في الطلب تحت القفل المفروض مسبقاً، بحيث لا يمكن لأى عملية مزامنة أخرى أن تقرأ أو تعدل نفس المخزون في تلك اللحظة.
4. سلامة العملية المالية الإجمالية: يُنشأ الطلب وسجل الدفع ضمن نفس الوحدة، مما يضمن أن كل عملية دفع بالمحفظة لها طلب مرتبط يمكن الرجوع إليه لاحقاً.

ملاحظة: يتضمن هذا الاختبار تأخيراً مقصوداً لمحاكاة الاتصال ببوابة دفع خارجية، وهو ما يتيح اختبار سيناريوهات التزامن الواقعية.

إعادة تخزين (Admin)

تم تطبيق المعاملات هنا مع Pessimistic Locking ، لأن عملية إعادة التخزين يجب أن تسلسل عند تنفيذها من قبل أكثر من مسؤول في

آن واحد، حتى لا تتعارض القراءة والكتابة فتفسد الإجمالي.

تضمن هذه الوحدة أن كل زيادة في المخزون مسجلة بدقة في سجل المخزون، وأنه في حال فشل التسجيل بلغى التعديل على المخزون تلقائياً.

يتم ذلك عبر عدة خطوات:

1. قفل صف المنتج في قاعدة البيانات بشكل حصري لضمان عدم تدخل أي عملية أخرى.
2. زيادة الكمية في المخزون وتحديث رقم الإصدار.
3. تسجيل عملية إعادة التخزين في سجل المخزون لأغراض التدقيق.

تحديث حالة الطلب Admin

تم تطبيق المعاملات هنا مع Pessimistic Locking لضمان أن تغيير حالة الطلب يحدث بشكل حصري دون تعارض مع أي عملية أخرى تتعامل مع نفس الطلب.

تضمن هذه الوحدة أن الحالة الجديدة تحفظ كاملة أو لا تحفظ على الإطلاق، وأنه لا يمكن لمسؤولين تعديل نفس الطلب في آن واحد.

يتم ذلك عبر عدة خطوات:

1. قفل صف الطلب في قاعدة البيانات بشكل حصري.
2. تعيين الحالة الجديدة وحفظها.

إنشاء تقييم للمنتج

تم تطبيق المعاملات هنا مع قيد قاعدة البيانات (Unique DB Constraint)، وليس عن طريق Optimistic Locking، إذ لا يوجد حقل إصدار في نموذج التقييم.

الحماية تأتي من قيد الفريدة على مستوى قاعدة البيانات الذي يُحدد أنه لا يمكن لمستخدم واحد أن يقيم نفس المنتج أكثر من مرة. إذا حاول مستخدمان إرسال تقييم لنفس المنتج في آن واحد، يفوز الأول ويحصل الثاني على رسالة خطأ واضحة.

يتم ذلك عبر عدة خطوات

1. محاولة إنشاء سجل التقييم الجديد.
2. التحقق قاعدة البيانات تلقائياً من أن هذا المستخدم لم يقيم هذا المنتج من قبل
3. في حال وجود تقييم سابق، توقف قاعدة البيانات العملية وترجع خطأ سلامة يُحول إلى رسالة واضحة للمستخدم.

ملاحظة: آلية التحكم التلقائي في الحفظ والتراجع

آلية الى Automatic Commit & Rollback:

عند استخدام transaction.atomic) نقد Django جميع التغييرات داخل معاملة قاعدة بيانات واحدة. إذا . اكتملت العملية بنجاح حتى النهاية، يتم commit لكل التغييرات معاً في لحظة واحدة. أما إذا حدث أي خطأ في أي خطوة داخل الوحدة، يتم rollback لكل التغييرات التي بدأت داخلها، بحيث لا تبقى أي حالة جزئية في قاعدة البيانات، وتعود إلى حالتها النظيفة قبل بدء العملية.

10.3 سيناريو كامل - مستخدم يشتري منتجاً

التوضيح آلية عمل المعاملات في النظام بشكل متكامل، نتابع كسيناريو توضيحي رحلة مستخدم يُضيف منتجاً للسلة ثم يكمل عملية الشراء، في بيئة يتنافس فيها مئة مستخدم على منتج لا يتبقى منه إلا عشر قطع.

المرحلة الأولى - إضافة المنتج للسلة

يضغط المستخدم على زر "أضف للسلة"، فيبدأ النظام بقراءة بيانات عنصر السلة وكمية المخزون المتاحة دون قفل مسبق بعدها يحاول تحديث صف السلة بشرط أن يكون رقم إصداره لم يتغير منذ القراءة. إذا نجح التحديث، تحفظ التغييرات وتكتمل العملية، وإذا فشل بسبب تدخل طلب آخر، تعاد العملية تلقائياً حتى تنجح أو تنتهي المحاولات.

المرحلة الثانية - إتمام الشراء

يضغط المستخدم على زر "إتمام الشراء"، فتبدأ أهم معاملة في النظام. يُرتَّب معرف المنتج بشكل ثابت ثم تقفل صفوف المنتجات في قاعدة البيانات بشكل حصري، مما يجعل بقية الطلبات المتزامنة تنتظر دورها تحت هذا القفل، يتحقق النظام من كفاية المخزون، ثم يخصمه ويسجل حركة المخزون، ثم ينشئ الطلب وينوده ويؤكد حالة الطلب، وينشئ سجل الدفع في خطوة واحدة، وأخيراً، تمسح السلة وتلغى البيانات المخزنة مؤقتاً. عند اكتمال كل هذه الخطوات، يرفع القفل وتعطى الفرصة للطلب التالي في الانتظار.

المرحلة الثالثة - ماذا يحدث للمستخدمين الآخرين؟

يأخذ المستخدم الثاني دوره بعد تحرر القفل فيجد أن المخزون نقص بمقدار واحد يستمر هذا الترتيب حتى تنتهي العشر قطع فيحصل بعدها كل مستخدم يحاول الشراء على رسالة تخبره بعدم كفاية المخزون، وتلفى معاملته دون أن تحدث أي تغيير في قاعدة البيانات النتيجة النهائية هي أن عشرة مستخدمين نجحوا، وتسعين . مستخدماً حصلوا على رسالة خطأ واضحة، ولم يتجاوز المخزون حدوده الصفرية في أي لحظة.

المرحلة الرابعة - تأكيد الدفع

بعد إنشاء الطلب، يكمل المستخدم عملية الدفع يقفل صف الطلب وصف الدفع بترتيب ثابت، ويتحقق النظام أولاً من أن هذا الطلب لم يدفع مسبقاً لمنع الدفع المزدوج. إذا كانت هذه أول محاولة دفع، تعين حالة الدفع إلى مكتمل وحالة الطلب إلى قيد المعالجة في نفس الوحدة، مما يضمن أنهما دائماً متزامنتان.

المرحلة الخامسة - Admin يعيد التخزين

يقرر المسؤول إعادة تخزين المنتج بإضافة خمسين قطعة. يقفل صف المنتج في قاعدة البيانات تضاف الكمية الجديدة، يحدث رقم الإصدار، ويُسجل التغيير في سجل المخزون عند اكتمال العملية، يرفع القفل ويصبح المنتج متاحاً للبيع من جديد.

المرحلة السادسة - Admin يلغي طلباً

يقرر المسؤول إلغاء طلب معين. يقفل صف الطلب أولاً، يتحقق النظام من أن حالته تسمح بالإلغاء، ثم تُعين حالته إلى ملفى وتحفظ التغييرات إذا كان الطلب قد شحن بالفعل، ترفض العملية وترجع رسالة ملائمة دون أي تعديل في قاعدة البيانات.

11. المتطلب 9 & 10 - اختبار الاستقرار تحت الضغط والقياس وتحديد الاختناقات

في هذا الاختبار تم استخدام JMeter لمحاكاة 100 مستخدم يقومون بتنفيذ Flow كامل يشبه سلوك مستخدم حقيقي داخل متجر إلكتروني. الهدف لم يكن اختبار Endpoint واحد فقط، بل اختبار تسلسل كامل يبدأ من تصفح المنتجات وينتهي بإنشاء طلب، الدفع، وكتابة تقييم.

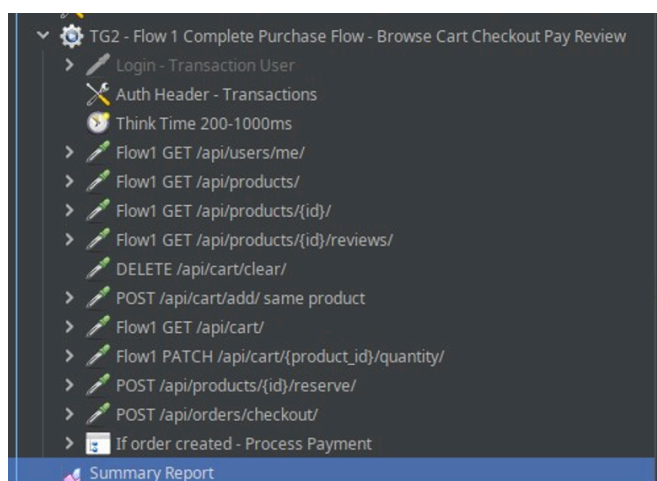
الـ Flow المستخدم:

1. GET /api/users/me/
2. GET /api/products/
3. GET /api/products/{id}/
4. GET /api/products/{id}/reviews/
5. GET /api/products/{id}/rating-summary/
6. DELETE /api/cart/clear/
7. POST /api/cart/add/
8. GET /api/cart/
9. PATCH /api/cart/{product_id}/quantity/
10. POST /api/products/{id}/reserve/
11. POST /api/orders/checkout/
12. GET /api/orders/{id}/
13. POST /api/orders/{id}/process-payment/
14. GET /api/orders/
15. POST /api/products/{id}/reviews/
16. GET /api/products/{id}/rating-summary/

وجود endpoint:

GET /api/orders/{id}/

داخل الـ flow ليس تكراراً. بعد تنفيذ POST /api/orders/checkout يقوم JMeter باستخراج order_id من response ثم يستخدمه لقراءة الطلب الذي تم إنشاؤه. الهدف من هذه الخطوة هو التأكد أن الطلب تم حفظه بشكل صحيح، وأن المستخدم يستطيع الوصول إليه بعد عملية checkout



11.1 بيئة الاختبار (Test Environment)

تم تنفيذ الاختبار على النظام في حالة AFTER، أي بعد تطبيق التحسينات السابقة مثل:

- Redis Cache
- Redis Distributed Lock
- Database Locking
- PgBouncer
- Celery Queue

قاعدة البيانات تحتوي على بيانات Seeded تشمل مستخدمي اختبار، منتجات، مخزون، طلبات وتجهيزات لازمة لتشغيل Flow كامل. تم اختيار منتج واحد عالي المخزون لاختبار الضغط على الشراء والحجز بشكل واضح.

معلومات البيئة:

Testing Tool: Apache JMeter

Concurrent Users: 100 users

Test Type: Complete purchase flow stress test

Database: PostgreSQL

Cache: Redis Cache باستخدام Cache-Aside Pattern

Queue: Celery + Redis Broker

Load Balancer: Nginx Load Balancer

Application Instances: 5 Django App Containers

Database Connection Pool: PgBouncer

Monitoring: Prometheus + Grafana + Exporters

11.2 نتائج الاختبار (Stress Test Result)

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Flow1 GET /api/users/me/	100	10	4	74	10.60	0.00%	5.0/sec	2.35	1.21	477
Flow1 GET /api/products/	100	23	7	98	17.43	0.00%	4.9/sec	443.02	1.18	91874
Flow1 GET /api/products/{id}/	100	8	4	46	6.74	0.00%	4.8/sec	2.87	1.17	610
Flow1 GET /api/products/{id}/reviews/	100	56	5	225	54.59	0.00%	4.7/sec	22.85	1.19	4937
DELETE /api/cart/clear/	100	10	5	43	6.62	0.00%	4.7/sec	1.69	1.24	368
POST /api/cart/add/ same product	100	19	10	59	10.11	0.00%	4.7/sec	2.46	1.38	533
Flow1 GET /api/cart/	100	11	5	42	7.26	0.00%	4.7/sec	2.54	1.10	554
Flow1 PATCH /api/cart/{product_id}/quantity/	100	17	8	61	10.21	0.00%	4.6/sec	2.38	1.31	529
POST /api/products/{id}/reserve/	100	13	6	44	6.62	0.00%	4.6/sec	2.33	1.40	510
POST /api/orders/checkout/	100	36	17	95	17.46	0.00%	4.6/sec	3.56	1.23	788
Flow1 GET /api/orders/{id}/	100	17	8	50	9.33	0.00%	4.7/sec	3.60	1.13	789
POST /api/orders/{id}/process-payment/	100	17	8	94	10.97	0.00%	4.7/sec	2.33	1.72	504
Flow1 GET /api/orders/	100	13	7	50	7.82	0.00%	4.8/sec	2.62	1.14	561
Flow1 POST /api/products/{id}/reviews/	100	16	8	73	10.37	0.00%	4.7/sec	2.56	1.75	559
Flow1 GET /api/products/{id}/rating-summary/ ...	100	9	3	73	8.44	0.00%	4.6/sec	1.86	1.20	411
TOTAL	1500	18	3	225	21.26	0.00%	50.4/sec	341.54	13.76	6934

تم تنفيذ الاختبار على 15 endpoint، وكل endpoint تم تنفيذه 100 مرة، أي أن إجمالي الطلبات كان 1500 requests

النتائج كانت مستقرة، ولم تظهر أى أخطاء أثناء الاختبار:

- Total Requests: 1500
- Failures: 0
- Error Rate: 0.00%
- Average Response Time: 18 ms
- Minimum Response Time: 3 ms
- Maximum Response Time: 225 ms
- Throughput: 50.4 requests/sec

هذه النتائج تعني أن النظام استطاع تنفيذ flow الشراء الكامل لـ 100 مستخدم بدون failures أو انهيار أو فقدان بيانات. كما أن زمن الاستجابة بقي منخفضاً جداً في أغلب العمليات، حيث كانت معظم endpoints ضمن عشرات المللي ثانية فقط.

أعلى endpoint من ناحية متوسط زمن الاستجابة كان

GET /api/products/{id}/reviews/

حيث سجل متوسطاً يقارب 56 ms، وهذا منطقي لأن هذا endpoint يرجع قائمة مراجعات المنتج، وبالتالي حجم البيانات المعادة أكبر من endpoints البسيطة مثل cart أو orders.

كذلك عملية:

POST /api/orders/checkout/

سجلت متوسطاً يقارب 36 ms، وهي نتيجة جيدة لأنها من أهم العمليات في النظام، حيث تتضمن قراءة السلة التحقق من المخزون، قفل المنتج خصم المخزون، إنشاء الطلب، وإنشاء سجل الدفع.

أما باقي العمليات مثل:

- GET /api/cart/
- POST /api/cart/add/
- PATCH /api/cart/{product_id}/quantity/
- POST /api/products/{id}/reserve/
- POST /api/orders/{id}/process-payment/

فقد بقيت بزمن استجابة منخفض وبدون أي أخطاء، وهذا يدل أن العمليات الحساسة في flow الشراء عملت بشكل صحيح تحت الضغط.

Result Interpretation 11.3

توضح نتائج الاختبار أن النظام قادر على خدمة 100 مستخدم متزامن يقومون بتنفيذ flow شراء كامل دون ظهور أخطاء. نسبة الفشل كانت 0%، وهذا يعني أن جميع الطلبات اكتملت بنجاح

كما أن متوسط زمن الاستجابة الكلي كان 18 ms فقط، وهو مؤشر جيد على أن التحسينات المطبقة سابقاً ساعدت النظام على التعامل مع الضغط خصوصاً:

- لتقليل القراءة المتكررة من قاعدة البيانات Redis Cache
- الحماية عمليات المخزون والطلبات Database locking

- لتخفيف ضغط اتصالات قاعدة البيانات PgBouncer
- لفصل الأعمال الخلفية عن مسار الطلب الأساسي Celery Queue
- Django containers لتوزيع الطلبات على عدة Nginx

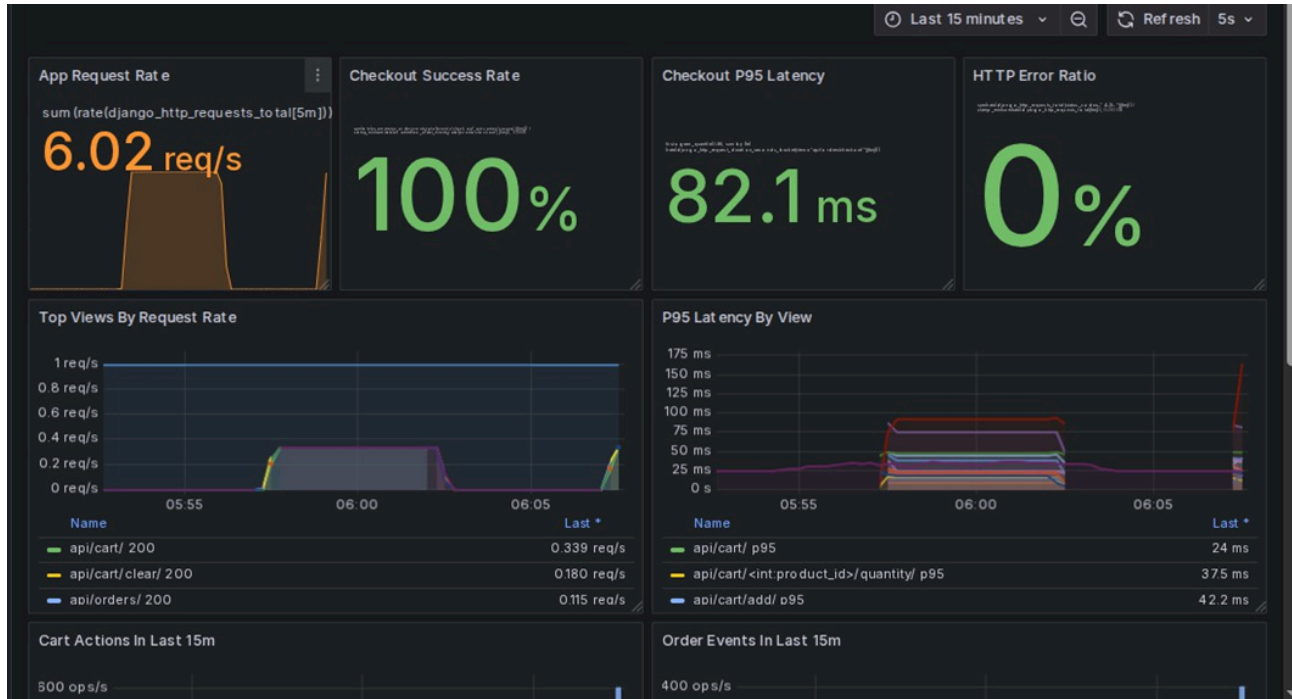
بشكل عام، نتيجة الطلب التاسع تثبت أن النظام في حالته المحسنة قادر على تنفيذ flow كامل لـ 100 مستخدم بنجاح، مع 0 أخطاء، وزمن استجابة منخفض، واستقرار واضح في العمليات الأساسية مثل cart, reserve, checkout, payment, review

11.4 تحليل نقاط الاختناق (Benchmarking and Bottleneck Analysis)

1-الهدف من هذا الطلب هو مراقبة النظام أثناء تنفيذ اختبار الضغط في الطلب التاسع وتحليل أداء أهم العمليات ثم تحديد ما إذا كان هناك اختناق واضح في النظام أم لا.

- Prometheus لجمع البيانات.
- Grafana لعرض الرسوم البيانية.
- cAdvisor لمراقبة containers.
- PostgreSQL Exporter لمراقبة اتصالات قاعدة البيانات.
- Redis Exporter لمراقبة Redis command rate.
- Django application metrics لمراقبة request rate, latency, status codes, checkout

2-Application Metrics and Endpoint Latency / مقاييس التطبيق وزمن استجابة ال endpoints



تعرض هذه الصورة مؤشرات أداء التطبيق:

- يوضح الرسم البياني معدل الطلبات في الثانية (Requests/sec) أثناء تدفق العمليات (Flow)، مع نجاح كامل لعمليات الدفع (Checkout) بنسبة 100%، ونسبة أخطاء HTTP تبلغ 0%. هذا يؤكد أن النظام يدير العمليات الأساسية بكفاءة ودون أي فشل أثناء الاختبار.
- كما يظهر أن زمن استجابة عمليات الدفع عند مستوى النسبة المئوية 95 (Checkout P95 Latency) كان 82.1ms، وهي

قيمة ممتازة لعملية حساسة تتضمن قراءة السلة، إنشاء الطلب، التعامل مع المخزون، وبدء المعاملة (Transaction) داخل قاعدة البيانات.

في الجزء السفلي من الصورة:

- تم عرض (Endpoints) الأكثر نشاطاً حسب معدل الطلبات، بالإضافة إلى زمن الاستجابة المتوسط وزمن استجابة P95 لكل منها.
- نلاحظ أن أعلى زمن استجابة كان في بعض عمليات السلة أو الدفع (Checkout)؛ نظراً لأنها تعتمد على الكتابة في قاعدة البيانات وإجراء معاملات تضمن سلامة البيانات.

خلاصة القول: توضح الصورة بشكل عام أن النظام كان مستقراً تماماً أثناء الاختبار، وأن عمليات الدفع تنجح بالكامل دون أخطاء، وبزمن استجابة مقبول للعمليات الحساسة.

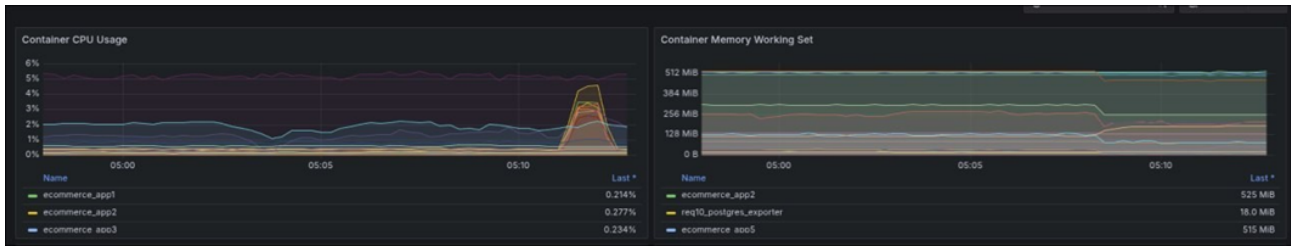
3. Business Events Monitoring / مراقبة أحداث العمل ومسارات النظام



توضح هذه الصورة أن الاختبار لم يكن مجرد قراءة ال (Endpoints)، بل شمل عمليات فعلية داخل النظام:

- يتضمن ذلك عمليات إضافة منتجات للسلة، تعديل الكمية، والدفع (Checkout).
- ظهور أحداث مثل checkout_success و payment_success يؤكد أن دورة الشراء الكاملة (Purchase Flow) قد تمت بنجاح.
- كذلك يوضح الرسم البياني لتوزيع حالات الاستجابة (HTTP Status Mix) أن الغالبية العظمى من الاستجابات كانت ناجحة، ولم تظهر أي أخطاء مؤثرة أثناء فترة الاختبار.

4. Infrastructure Monitoring / مراقبة البنية التحتية



توضح هذه الصور حالة موارد النظام أثناء تنفيذ اختبار الضغط (Stress Test):

- **معدل أوامر ريديس (Redis Command Rate):**
يظهر الرسم البياني أن معدل عمليات وأوامر Redis كان مستقرًا وعاليًا في البداية حول 7 ops/s إلى 9 ops/s (عملية في الثانية)، ثم ارتفع مؤقتًا إلى أكثر من 20 ops/s أثناء ذروة الاختبار. هذا الارتفاع طبيعي جداً لأنه يعكس استخدام Redis في الكاش (Caching) وقراءة البيانات المتكررة من الذاكرة بدلاً من الرجوع المستمر لقاعدة البيانات الأساسية.
- **استهلاك المعالج للحاويات (Container CPU Usage):**
يظهر الرسم البياني ارتفاعاً مؤقتاً في استهلاك المعالج (CPU) عند بدء الاختبار، وتحديدًا على حاويات التطبيق (Application Containers). هذا الارتفاع كان لفترة قصيرة ثم عاد إلى مستواه الطبيعي، مما يدل على أن النظام تعامل مع الضغط بكفاءة ودون استمرار في استهلاك المعالج أو وصوله إلى حالة التشبع الكامل (Saturation).
- **حجم الذاكرة العاملة للحاويات (Container Memory Working Set):**
يوضح الرسم البياني أن استهلاك الذاكرة (Memory) كان مستقرًا تقريباً طوال فترة الاختبار، ولم يظهر أي نمو تدريجي مستمر. هذا يعتبر مؤشراً ممتازاً على عدم وجود أي تسريب للذاكرة (Memory Leak) أو تراكم غير طبيعي في البيانات أثناء تشغيل دورة الشراء الكاملة.

Bottleneck Analysis-5 / تحليل الاختناقات

بناءً على النتائج، لا يوجد أي اختناق (Bottleneck) حرج يتسبب في انهيار النظام أو فشل الطلبات:

- نسبة الأخطاء كانت 0%.
- جميع عمليات الدفع وال Checkout نجحت بالكامل.
- النظام حافظ على استقراره طوال فترة الاختبار.

لكن، يمكن اعتبار نقطة الضغط الأساسية (أو الأبطأ نسبياً) هي:

`/POST /api/orders/checkout`

هذه ال (Endpoint) هي أثقل جزء في تدفق العمليات (Flow)؛ لأنها لا تقوم بمجرد قراءة بسيطة للبيانات، بل تدير عملية مركبة تشمل قراءة السلة، إنشاء الطلب، التحقق من المنتجات، خصم المخزون، إنشاء عناصر الطلب، وتجهيز عملية الدفع. لذلك، تعتمد هذه العملية بشكل

كبير على المعاملات (Transactions) والقفل التشاؤمي (Pessimistic Locking) لحماية المخزون ومنع البيع الزائد (Overselling).

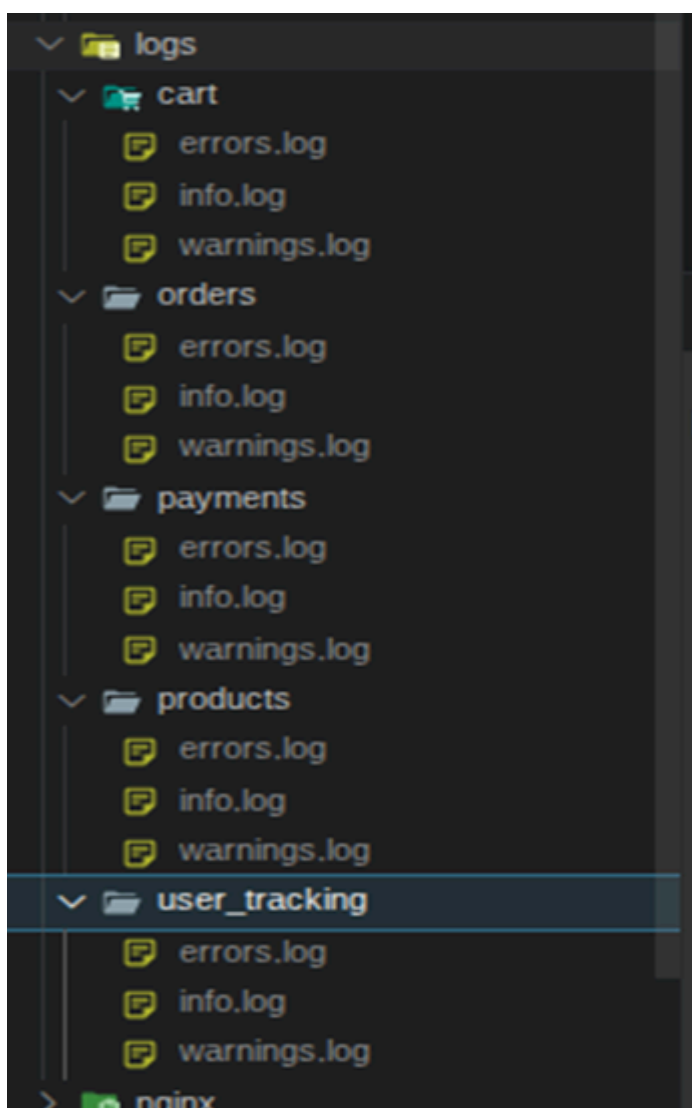
ملاحظة: الضغط هنا أمر متوقع وطبيعي وليس خطأ في النظام؛ لأن النظام يفضل سلامة البيانات وسياقها الصحيح (Data Integrity) على السرعة المطلقة. لو تم إزالة الآليات الحمائية (مثل القفل والـ Transactions)، لأصبح الأداء أسرع ظاهرياً، ولكن ذلك قد يؤدي إلى مشاكل خطيرة مثل بيع كميات غير متوفرة في المخزن الحقيقي.

الاستنتاج (Conclusion)

لا يوجد اختناق حرج أو خطير في الاختبار الحالي، ولكن النقطة الأكثر حساسية وقابلية للمراقبة المستمرة هي عملية الـ **Checkout**؛ نظراً لأنها تجمع بين معاملات قاعدة البيانات، قفل المخزون، وإنشاء الطلبات (Database Transaction, Stock Locking, Order Creation).

Logs Evidence-6 / أدلة سجلات النظام

تم تفعيل سجلات النظام (Logs) بشكل مسمّى ومصنف حسب أجزاء النظام؛ ليسهل تتبع مسار الطلبات وفحص السلوك بدقة أثناء فترة الاختبار.



شرح الصورة: سجلات النظام (Logs Structure)

توضح الصورة أن ملفات سجلات النظام (Logs) مقسمة ومُنظمة بشكل مسمّى ومصنف حسب أجزاء النظام المختلفة، مثل: **cart** (السلة)، **orders** (الطلبات)، **payments** (المدفوعات)، **products** (المنتجات)، و **user_tracking** (تتبع المستخدمين). هذا التقسيم يجعل تتبع تدفق العمليات (Flow) أسهل بكثير؛ لأن كل جزء من النظام يسجل الأحداث الخاصة به في ملف مستقل.

1. مجلد تتبع المستخدمين (/logs/user_tracking)

- **info.log**: يسجل كل طلب (Request) ناجح مع معلومات تفصيلية مثل معرف الطلب (**request_id**)، معرف المستخدم (**user_id**)، النهاية الطرفية المستهدفة (**endpoint**)، ورمز حالة الاستجابة (**status_code**). هذا يساعد على تتبع رحلة المستخدم بالكامل داخل النظام من بداية الطلب وحتى نهايته.
- **warnings.log**: يسجل الطلبات التي نجحت بالفعل ولكنها استغرقت زمناً أعلى من الحد المتوقع والمسموح به (**elapsed_ms**). وجود تحذيرات قليلة هنا لا يعني فشل النظام، بل يساعد المطورين على معرفة أي (Endpoints) أصبحت أبطأ تحت الضغط لتطويعها مستقبلاً.
- **errors.log**: مخصص للأخطاء الحرجة مثل أخطاء الخادم الداخلية (Error 500) أو الاستثناءات البرمجية (Exceptions). بقاء هذا الملف فارغاً تماماً أثناء الاختبار يعد دليلاً قاطعاً على أن النظام لم يتعرض لأي انهيار أو أخطاء داخلية.

2. سجلات أجزاء النظام الأخرى

- **logs/cart/info.log**: يؤكد أن المستخدمين قاموا بعمليات السلة الفعلية مثل: جلب بيانات السلة (**get cart**)، الإضافة للسلة (**add to cart**)، تعديل الكمية (**update quantity**)، ومسح السلة (**clear cart**).
- **logs/orders/info.log**: يوضح أن عمليات إنشاء الطلبات تمت بنجاح، وهذا يؤكد مجدداً أن الاختبار شمل عمليات كتابة فعلية وحقيقية في قاعدة البيانات وليس مجرد قراءة.
- **logs/payments/info.log**: يظهر أن عمليات الدفع تمت بنجاح مباشرة بعد إنشاء الطلبات، مما يؤكد اكتمال دورة الشراء الكاملة (Purchase Flow).
- **logs/products/info.log**: يسجل أحداث المنتجات المصاحبة للعمليات مثل: حجز المنتجات، خصم المخزون، وإصدار التقييمات.

هذه البيئة هي بيئة تجريبية مبنية باستخدام Docker Compose، وليست بيئة تشغيلية نهائية ذات إتاحة عالية (High-availability production deployment).

القيود الحالية:

- يُعد Nginx خدمة وكيل عكسي مفردة (Single reverse proxy service).
 - يُعد PostgreSQL خدمة قاعدة بيانات مفردة.
 - يُعد Redis خدمة مفردة.
 - يُعد PgBouncer خدمة تجميع مفردة (Single pooling service).
 - عدد حاويات التطبيق حالياً هو خمسة: app1، و app2، و app3، و app4، و app5.
 - تعتمد نتائج الاختبار على موارد الجهاز المحلي (Local machine resources).
- لا تُبطل هذه القيود المتطلبات الخمسة الأولى؛ لأن الهدف الأساسي هو إثبات المزامنة (Synchronization)، والسعة المحكومة (Controlled capacity)، والطوابير غير المتزامنة (Asynchronous queues)، والمعالجة المجزأة (Chunked processing)، وتوزيع الأحمال (Load distribution).
-